Deployment, CI/CD, Observability & Tooling — From Scratch

Mapped to your GitLab resume · built from your office drills (Oct 4–7) · with a full grill section

Prepared 2026-10-07 for the Oct 9 round (Karan Bhatia — identity/IAM, multi-tenancy, observability at scale).

Contents

- Deployment, CI/CD, Observability & Tooling — From Scratch
  - Contents
  - How to use this document
  - Resume → chapter map
  - Chapter 0 — The big picture in one page
- Part 1 — Packaging and running code
  - Chapter 1 — Containers and images (Docker)
  - Chapter 2 — How a service actually runs (process model, workers, resilience)
  - Chapter 3 — Kubernetes
  - Chapter 4 — AWS: the pieces that matter for an agent platform
  - Chapter 5 — Data layer: Redis, the database, Data Cloud, event-driven
  - Chapter 6 — CI/CD
- Part 2 — Seeing, controlling and securing it
  - Chapter 7 — Observability
  - Chapter 8 — The agent's tools (actions, discovery, MCP)
  - Chapter 9 — Engineering tooling: evals, harnesses, triage
  - Chapter 10 — Identity and security across the stack (his home turf)
- Part 3 — The Grill
  - Round 1 — The loop and where it runs
  - Round 2 — Containers and runtime
  - Round 3 — Kubernetes
  - Round 4 — AWS
  - Round 5 — CI/CD
  - Round 6 — Observability and on-call
  - Round 7 — Tools, MCP and evals
  - Round 8 — Identity, data and design curveballs
  - Appendix — Rapid-fire glossary

How to use this document

Every chapter has the same four parts:

1. **From scratch** — the concept explained with no assumed knowledge. Read this until you could teach it.
2. **In your system** — what Agentforce for Setup actually does, from the office drills.
3. **Say it** — a spoken version (45–90 s). Practise out loud.
4. **Where he'll push** — the follow-ups a principal-level interviewer asks, with answers.

Part 3 is the **grill**: 58 interview questions in rounds of increasing difficulty, each with what he's testing, a model answer, a follow-up, and the trap to avoid.

Evidence tags (so you never overclaim)

Tag	Meaning	How to speak about it
[✓]	Verified in code or config by office Claude	State it as fact
[~]	From a Slack digest, a doc, or only the first drill pass	"My understanding is…"
[✗]	Searched for and **not found**	Don't state it. Reason from first principles instead
(no tag)	General industry concept	Teach it confidently

All "Say it" answers use generic names (planner service, reasoning service, Agent API). That's how you speak in an interview — never internal service names, org IDs or channel names.

Ownership language

Say **"I"** for what you designed or built (graph, skills, approval flow, actions, evals). Say **"we"** or **"the platform"** for clusters, base images, pipeline framework, gateways and Data Cloud. A Fellow-level interviewer respects precise ownership; an overclaim gets found with one follow-up question.

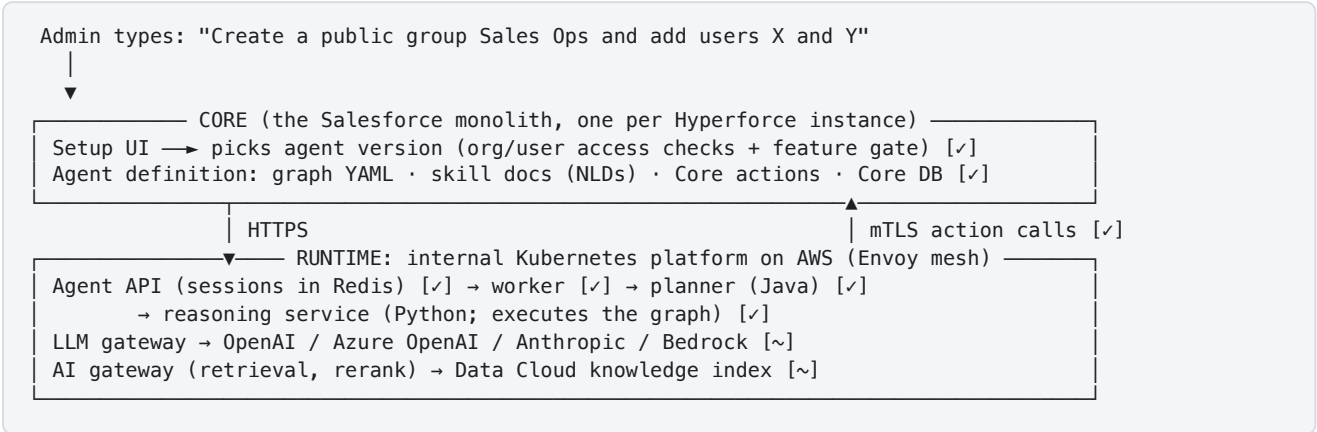
Resume → chapter map

Resume words	Where it's explained	Risk
"Deployed and operated agent services on **AWS**"	Ch 4 (AWS), Ch 3 (Kubernetes)	High — most details [✗]; use first-principles answers
"**Docker**"	Ch 1 (Containers)	Medium — reasoner image [✓]
"**Kubernetes**"	Ch 3	High — platform-owned; LLM-gateway sizing [~]
"**CI/CD**"	Ch 6	Medium — two-lane model [✓/~]
"**Splunk observability** … distributed tracing, structured logs, CI signals"	Ch 7	Medium — log records [✓], gaps known
"**event-driven infrastructure**"	Ch 5.4	Medium
"throughput up **~40%** with stronger **fault isolation**"	Ch 2.6 (bulkheads)	**Number [✗]** — mechanism only unless you know the source
"**on-call rotation** ownership"	Ch 7.8	High — paging details [✗]

Resume words	Where it's explained	Risk
"permission-aware tools … **MCP, REST APIs, Metadata API, Tooling API**"	Ch 8, Ch 10	Low–medium
"onboarded **eight capabilities to Headless 360** … tool discovery, MCP execution"	Ch 8.4–8.5	Count [✗]
"**human-in-the-loop** … privileged actions run under the **requesting user's own permissions**"	Ch 10, Ch 8.3	**User-id header [✗] — verify**
"**AI evaluation framework** … replayed production failures"	Ch 9	Medium
"**AI Bug Triage** skill … Splunk logs, repository context"	Ch 9.5	Low
"**ResolveFlow** … LangGraph interrupts, Postgres, GitHub OAuth"	Ch 5.2, Ch 10.4	Your own repo — hands-on
"**AdaptiveRAG** … Lambda, API Gateway"	Ch 4 box	Your hands-on AWS story, if you built it
"**AutoRABIT** … release-management and CI/CD workflows"	Ch 6 box	Know how "~30%" was measured

Chapter 0 — The big picture in one page

Before any detail, hold this picture. Everything else hangs off it.



Three sentences that organise the whole document:

1. **Two places, two release lanes.** The agent's *definition* (graph, skills, actions) lives in Core and ships with the Core release. The *runtime* is microservices in containers on Kubernetes, each with its own pipeline. (Ch 1–3, 6)
2. **Two identities on every write.** The *service* proves who it is with an mTLS certificate; the *user* travels as context so Core checks the admin's own permissions. (Ch 10)
3. **One request, many hops, many turns.** A request with an approval is several HTTP turns across several services, so observability lives or dies on correlation IDs. (Ch 7)

Part 1 — Packaging and running code

Chapter 1 — Containers and images (Docker)

1.1 From scratch

The problem. Code that runs on your laptop fails in production because something differs: Python version, a system library, an environment variable. Containers fix this by shipping the code *together with* its whole runtime environment.

What a container really is. A container is not a small virtual machine. It's an ordinary Linux process that the kernel isolates in two ways:

- **Namespaces** limit what the process can *see*: its own process list (PID namespace), its own network interfaces (network namespace), its own filesystem view (mount namespace), its own users (user namespace).
- **cgroups** (control groups) limit what it can *use*: CPU time, memory, I/O.

Because containers share the host's kernel, they start in milliseconds and cost almost nothing extra. A VM boots a whole operating system.

Image vs container.

- An **image** is a read-only template: a filesystem snapshot plus metadata (which command to run, which port, which user).
- A **container** is a running instance of an image: the image's layers plus a thin writable layer plus a live process.
- One image → many identical containers. That's why scaling works.

Layers. An image is built from a **Dockerfile**. Each instruction (`RUN`, `COPY`) creates a **layer**. Layers are cached and shared between images.

```
FROM python:3.12-slim              # base image (layer 1)
WORKDIR /app
COPY pyproject.toml uv.lock ./     # dependency manifest first…
RUN uv sync --frozen               # …so this expensive layer is cached
COPY src/ ./src/                   # code changes only bust from here down
USER 1001                          # don't run as root
EXPOSE 8128
CMD ["gunicorn", "-c", "gunicorn_config.py", "app:app"]
```

Order matters: put things that rarely change (dependencies) *above* things that change every commit (your code). Otherwise every code change reinstalls every dependency.

Base images. The `FROM` line. Smaller ("minimal", "slim", "distroless") means fewer packages, fewer known vulnerabilities (CVEs), faster pulls. Large companies maintain **golden base images** that a platform team patches centrally — service teams just bump the version.

Multi-stage builds. A build needs compilers and build tools; the running service doesn't. A multi-stage Dockerfile builds in one stage and copies only the result into a clean runtime stage. Result: a smaller image with less attack surface.

Non-root user. If an attacker exploits your app, they get whatever user the process runs as. Running as a non-root UID (e.g., 1001) means they don't start as root inside the container. Kubernetes can enforce this (`runAsNonRoot: true`).

Registry, tags and digests.

- A **registry** stores images (AWS ECR, Artifactory, Nexus, internal registries).
- A **tag** (`v1.4.2` , `latest`) is a *mutable* label — someone can re-point it.
- A **digest** (`sha256:9f2c…`) is the *immutable* content hash.
- Production should deploy an immutable reference: a digest or a tag that's never reused (e.g., the git SHA).

Scanning, signing, provenance.

- **Scanning** checks OS packages and libraries against vulnerability databases; a policy blocks release on, say, any "critical".
- **Signing** (e.g., cosign) lets the cluster verify an image really came from your CI.
- **Provenance** (SLSA attestations) records *how* it was built.

Build once, promote everywhere. The exact image (same digest) that passed staging is the one deployed to production. Rebuilding per environment means production runs something you never tested.

1.2 In your system

Item	Value	Tag
Reasoning service base image	Corporate RHEL 9 + minimal Python 3.12, pinned version	[✓]
Dependency install	`uv` mounted from the matching *build* image at build time — same goal as multi-stage: no build tooling in the runtime image	[✓]
User	Non-root `appuser` , UID/GID 1001	[✓]
Port	8128	[✓]
Java services (Agent API, worker, planner)	Built by internal CI into Docker images	[~]
Image size, scan severity that blocks, signing, same-digest promotion	—	[✗]

1.3 Say it

> "Each runtime service ships as a container image. Take the reasoning service: it's built on our corporate RHEL 9 minimal Python 3.12 image, which a platform team patches centrally. Dependencies are installed with `uv` from a matching build image, so the runtime image carries no compilers or build tools — smaller and less attack surface. It runs as a non-root user. The image is built once by CI, pushed to an internal registry, and the same artifact is promoted through environments."

1.4 Where he'll push

- **"Why minimal base images?"** Fewer packages → fewer CVEs to patch, smaller pulls, faster scale-out.
- **"Why does non-root matter if it's already in a container?"** Containers share the host kernel. A container escape as root is far worse than as an unprivileged user. Defence in depth.
- **"How do you know prod runs what you tested?"** Immutable references (digest or SHA tag), promote the same artifact, ideally signature verification at admission. *Honest:* "Whether our pipeline verifies signatures at admission I haven't confirmed."
- **"Image layer caching — why does order matter?"** Covered above.

Chapter 2 — How a service actually runs (process model, workers, resilience)

2.1 From scratch: concurrency models

A web service is a process listening on a port. The question is how it handles many requests at once:

Model	How it works	Good for	Weakness
Multi-process (pre-fork)	A master process forks N worker processes; each handles requests	Using all CPU cores; isolation (a crash kills one worker)	Memory × N
Multi-threaded	One process, many threads (e.g., Java/Tomcat thread pool)	Blocking I/O in Java	Threads are heavy; pool exhaustion
Async event loop	One thread juggles thousands of waiting requests; while one waits on the network, another runs	**I/O-bound work** — like waiting on an LLM	CPU-heavy code blocks everyone

Python specifics. Python's GIL means threads can't run Python code in parallel on multiple cores. So Python services combine **processes** (for cores) with **async** (for I/O):

- **FastAPI** — the web framework (ASGI = async interface).
- **Uvicorn** — the ASGI server running the event loop.
- **Gunicorn** — the process manager: a master that forks N workers, restarts dead ones, handles signals.
- `gunicorn -k <UvicornWorker>` = N processes × one event loop each → all cores *and* async.

2.2 Why LLM services are I/O-bound (the key insight)

An agent turn spends most of its time **waiting**: on the LLM (seconds to tens of seconds), on retrieval, on Core. While waiting, it uses almost no CPU. Consequences you should say out loud:

- One async worker can hold **hundreds of concurrent turns**.
- **CPU is the wrong scaling signal** — it stays low even when the service is saturated. Scale on **in-flight requests** or connection count instead.
- The real limits are **memory per worker, open connections, and downstream rate limits** (LLM tokens per minute).

2.3 Fork and copy-on-write (`preload_app`)

Gunicorn can load your app *once* in the master and then fork workers (`preload_app = True`). Forked workers share the master's memory pages until they write to them (**copy-on-write**) — saving memory.

The catch: objects holding live OS resources don't survive a fork. An async HTTP client created before the fork has sockets and an event loop that every worker would now share. Results: corrupted connections, hangs, "event loop is closed" errors.

So: keep `preload_app = False` and let **each worker build its own clients after it starts**.

2.4 Worker recycling (`max_requests` + jitter)

Long-running Python processes slowly grow memory (fragmentation, small leaks in libraries). `max_requests = 5000` restarts a worker after 5000 requests. `max_requests_jitter = 100` adds a random 0–100 so **all workers don't restart at the same moment** (which would briefly drop capacity to zero — a self-inflicted thundering herd).

2.5 Graceful shutdown and draining

When a deploy replaces a pod, the process gets **SIGTERM** ("please stop") and, after a grace period, **SIGKILL** ("stop now"). Graceful shutdown means:

1. Stop accepting new requests.
2. Let in-flight requests finish (drain).
3. Exit before SIGKILL.

For long-lived connections (streaming responses, WebSockets) the drain can take minutes. **The timeouts must nest — each inner one shorter than the one outside it:**

```
turn drain deadline  <  gunicorn graceful_timeout  <  gunicorn timeout  <  Kubernetes
terminationGracePeriodSeconds
```

If any outer timeout is *shorter* than an inner one, SIGKILL lands mid-turn and the user's stream breaks.

2.6 Resilience patterns (and "throughput with fault isolation")

Pattern	What it does	Rule of thumb
Timeout	Stop waiting after N seconds	Every network call has one. Inner < outer.
Retry with exponential backoff + jitter	Try again after 100 ms, 200 ms, 400 ms… plus randomness	Only retry **idempotent** operations (or ones with a dedupe key). Cap attempts.
Circuit breaker	After N failures, stop calling the dependency for a while ("open"), then probe ("half-open")	Fail fast instead of piling up waiting requests
Bulkhead	Separate pools (threads, connections, workers, queues) per dependency or per workflow	One slow dependency can't exhaust everything — **this is "fault isolation"**
Rate limiting / load shedding	Reject excess work early (HTTP 429)	Protect yourself and downstreams; per-tenant fairness
Deadline propagation	Pass "you have 40 s left" down the call chain	Better than fixed per-hop timeouts

On your resume's "throughput up ~40% with stronger fault isolation": the mechanisms that produce that kind of result are bulkheads per workflow, moving blocking calls to async, right-sizing workers, and removing a shared bottleneck. **The number itself was [✗] in both drills.** Explain the mechanism; only state the number if you can state how it was measured (unit, window, same resources).

2.7 In your system

Item	Value	Tag
Reasoning service	Python, gunicorn + custom *draining* Uvicorn worker, FastAPI	[✓]
Workers	`WORKER_COUNT` env, default **6**	[✓]
`preload_app`	**False** — comment says copy-on-write breaks the async HTTP client	[✓]
`max_requests`	**5000**, jitter **100**	[✓]
Drain chain (voice/WebSocket path)	turn drain 320 s + post-connection 300 s = **620** < graceful **640** < timeout **700** < Kubernetes grace (asserted < **902**)	[✓] comment
Text SSE stream on SIGTERM	—	[✗]
Core → gateway calls	2 attempts in prod, fixed 3 s wait, **no circuit breaker on the Core side**	[~]
LLM gateway	Retries; circuit breaker fails OpenAI over to Azure; kill switch to another region	[~]
MCP proxy	60 s timeout × `RETRY_COUNT=2`, 3-minute mesh ceiling	[✓] spec
Token-fetch client (local/dev)	90 s	[✓]
A known symptom	Long turns (~4.5–5 min) show "Something went wrong" although the change succeeded — the timeout chain doesn't line up	[~]

2.8 Say it

> "The reasoning service is gunicorn with async Uvicorn workers — six by default. An agent turn is mostly waiting on the model, so async lets one worker hold many turns at once, and processes give us all the cores. Two settings I like explaining: we don't preload the app in the master, because forking after creating an async HTTP client breaks its connections — each worker builds its own. And workers recycle after about 5000 requests with jitter, so memory stays bounded and they don't all restart together. For deploys, the drain timeouts nest inside each other so in-flight turns finish before Kubernetes kills the pod."

2.9 Where he'll push

- **"What happens when the LLM provider slows down to 60 s per call?"** In-flight turns pile up; async absorbs it until memory or connection limits; then timeouts fire. Without a circuit breaker you keep sending traffic into a slow dependency. Answer: per-dependency timeout + breaker + fail over (the gateway does provider failover) + shed load with a clear message.
- **"Your retries — are they safe?"** Only if the operation is idempotent. An agent turn that already executed a write is *not* safe to blindly retry. That's why writes need dedupe keys.
- **"Inner timeout greater than outer — what happens?"** The outer layer gives up and reports failure while the inner work continues and may succeed — the user sees an error for a change that actually happened. (Exactly the ~5-minute symptom.)

Chapter 3 — Kubernetes

3.1 From scratch: why it exists

You have dozens of services, each needing several copies, across hundreds of machines. Someone has to decide which machine runs what, restart crashed copies, add copies under load, route traffic, and roll out new versions without downtime. **Kubernetes** does that.

It's declarative. You don't say "start 3 containers". You submit **desired state** ("there should be 3 replicas of image X") and **controllers** continuously make reality match it (*reconciliation*). Kill a pod — the controller notices 2 ≠ 3 and starts another.

3.2 The parts

Cluster

- **Control plane:** API server (the front door — everything is an API call), etcd (the database of desired state), scheduler (picks a node for each pod), controllers (reconcile loops).
- **Worker nodes:** kubelet (runs pods on the node), container runtime, networking proxy.

Workload objects

Object	What it is
Pod	Smallest unit: one or more containers sharing an IP and volumes. Sidecars (e.g., an Envoy proxy, a Trust Layer filter) live in the same pod.
Deployment	"Keep N replicas of this pod template, and roll out changes gradually." Manages ReplicaSets.
Service	A stable DNS name + virtual IP that load-balances to all ready pods matching a label. Pods come and go; the Service name doesn't.
Ingress / Gateway	HTTP routing from outside the cluster to Services.
Namespace	A folder for objects; scope for access control (RBAC) and quotas.
ConfigMap / Secret	Configuration injected as env vars or files. Production secrets usually come from an external manager (Vault, AWS Secrets Manager) via a sync controller or CSI driver.

3.3 Resources: requests, limits, QoS

- **Request** = what the scheduler *guarantees* the pod (used to place it on a node).
- **Limit** = the hard ceiling.
- **CPU above the limit → throttled** (slowed down, not killed). Shows up as latency.
- **Memory above the limit → OOMKilled** (the container is killed and restarted).
- **QoS classes:** *Guaranteed* (request = limit — last to be evicted), *Burstable* (request < limit), *BestEffort* (none set — first evicted).

3.4 Probes

Probe	Question it answers	On failure
Liveness	"Is this process stuck?"	Restart the container
Readiness	"Can it take traffic right now?"	Remove from the Service's endpoints (no restart)
Startup	"Has it finished starting?"	Holds off liveness so slow starts aren't killed

Classic mistake: a liveness probe that checks a downstream dependency. When the dependency blips, *every* pod restarts at once → a self-made outage. Liveness checks the process only; readiness may check critical dependencies.

3.5 Pod termination (what happens during a deploy)

1. Pod marked *Terminating*; removed from Service endpoints (traffic stops — eventually).
2. `preStop` hook runs (often a short sleep, because endpoint removal propagates asynchronously).
3. **SIGTERM** sent to the container.
4. App drains in-flight work.
5. After `terminationGracePeriodSeconds` (default **30 s**) → **SIGKILL**.

For agent turns lasting minutes, 30 s is far too short — hence your ~900 s grace and the nested drain chain (Ch 2.5).

3.6 Scaling and availability

- **Horizontal Pod Autoscaler (HPA):** adjusts replica count from a metric. Formula: `desired = ceil(current × currentValue / target)`. Metrics can be CPU, memory, or **custom/external** (in-flight requests, queue depth — KEDA makes event-driven scaling easy).
- **Stabilization window:** HPA waits before scaling *down* (default 300 s) to avoid flapping. For long agent turns, make it longer than your longest turn.
- **PodDisruptionBudget (PDB):** "never voluntarily take me below N available pods" — protects you during node drains and cluster upgrades.
- **Topology spread / anti-affinity:** spread replicas across zones and nodes, so losing one AZ doesn't take out the service.

3.7 Helm (packaging Kubernetes YAML)

Raw Kubernetes YAML repeats a lot across environments. **Helm** packages it as a **chart**: templates + a `values.yaml`.

- Environment differences live in **layered values files**: base → environment → region/instance → domain. The later file overrides the earlier.
- `helm upgrade` deploys a new release; `helm rollback` returns to a previous revision.

3.8 Service mesh (Envoy)

A **service mesh** puts a proxy (Envoy) next to every pod as a sidecar. All traffic in and out goes through it, which gives you — without changing app code:

- **mTLS** between services (encryption + service identity),
- retries, timeouts, circuit breaking at the network layer,
- uniform telemetry (request rates, latencies),
- traffic shifting (send 5% to the new version).

A control plane distributes certificates and routing config to all proxies.

3.9 GitOps and progressive delivery (Argo)

- **Argo CD (GitOps):** the desired cluster state lives in Git; a controller syncs the cluster to Git and flags **drift** (manual changes).
- **Argo Rollouts:** replaces the plain rolling update with **canary** or **blue/green** steps — shift 5% → pause → run an **analysis** (query error rate/latency) → continue or **automatically roll back**.

3.10 In your system

Item	Value	Tag
Platform	Salesforce's internal Kubernetes platform on AWS (Hyperforce). Services deployed per *instance*, grouped into *functional domains* (e.g., a bots domain, an AI domain)	[~]
Helm values layering	base → environment → instance → domain	[~]
Mesh	Inside a domain: HTTP upgraded to **mTLS by the Envoy mesh**. Across domains: front door / ingress-egress gateways	[~]
Mesh addresses	Planner on mesh port 7442; worker 7444; reasoner 8128	[✓]
LLM gateway	HPA **3–9 replicas**, **4 CPU / 12 GiB**, Trust Layer **sidecar**	[~]
AI (retrieval) gateway	HPA **3–12 replicas**	[~]
Reasoner requests/limits, probes, PDB, HPA metric	—	[✗]
One service's platform config	Applied by hand with a CLI, **not by CI** → drift risk	[✓] README

3.11 How you'd size and scale the reasoning service (first principles — use when asked)

1. **Scaling signal:** in-flight turns per pod (or event-loop concurrency), not CPU. Target ~60–70% of what a pod can safely hold.
2. **Min replicas:** enough that losing one AZ still leaves capacity for peak.
3. **Scale-down stabilization:** longer than the longest turn so the autoscaler doesn't kill pods mid-conversation.
4. **Memory:** measure per-worker steady-state memory × worker count + headroom; set request = limit for memory (avoids OOM surprises and gives Guaranteed QoS for memory).
5. **CPU:** modest request, generous or no limit (throttling adds latency to an already slow path).
6. **PDB:** `maxUnavailable: 1` (or a %), so node drains don't take capacity out in bulk.
7. **Graceful termination:** grace period > drain chain; readiness flips false immediately on SIGTERM.
8. **Spread** across AZs with topology spread constraints.

3.12 Say it

> "The runtime services run on our internal Kubernetes platform on AWS. Each service is a Deployment packaged with Helm, with values layered from base to environment to instance, and they sit in a service mesh — Envoy sidecars give us mTLS between services plus uniform telemetry. The platform team owns the clusters; service teams own their chart values and alerts. For an LLM-bound service like ours, the interesting decision is scaling: CPU is the wrong signal because a worker waiting on the model uses almost none, so you scale on in-flight turns, keep scale-down slow so you don't kill long turns, and set the termination grace period longer than the drain chain."

3.13 Where he'll push

- **"OOMKilled vs throttled?"** Memory over limit → killed; CPU over limit → slowed.
- **"Liveness vs readiness?"** Restart vs remove-from-traffic; never let liveness depend on downstreams.
- **"How does a rolling update avoid dropping requests?"** Readiness gates new pods, maxUnavailable keeps capacity, preStop + SIGTERM drain old pods.
- **"What did *you* configure?"** Be precise. If it's the platform: "The platform owns cluster config; I can walk you through how I'd set it for our workload" → §3.11.

Chapter 4 — AWS: the pieces that matter for an agent platform

4.1 From scratch: geography and boundaries

- **Region** — a geographic area (us-east-1, us-west-2). Data stays in a region unless you move it. *Data residency* questions are region questions.
- **Availability Zone (AZ)** — a separate data centre (or group) inside a region with independent power and network. Spreading across **≥ 3 AZs** survives losing one.
- **Account** — the hard security and billing boundary. Big companies use many accounts (per environment, per team, per cell).

4.2 Networking (VPC)

Piece	What it does
VPC	Your private network inside AWS (an IP range).
Subnets	Slices of the VPC, each in one AZ. **Public** subnets route to the internet; **private** subnets don't. Services live in private subnets.
Internet gateway / NAT gateway	Internet gateway lets public subnets talk to the internet. NAT lets private subnets make *outbound* calls (e.g., to OpenAI) without being reachable from outside.
Security groups	Stateful firewalls on each instance/pod ENI: "allow 443 from the load balancer only".
Load balancers	**ALB** = layer 7 (HTTP routing by path/host, TLS termination). **NLB** = layer 4 (TCP, very high throughput, static IPs).
VPC endpoints / PrivateLink	Reach AWS services (S3, Bedrock) or another company's service **privately**, without traversing the internet. Important for data-handling guarantees.

4.3 Compute choices — the trade-off table

	Lambda	ECS (Fargate)	EKS (Kubernetes)
Unit	Function invocation	Container task	Pod
Max run time	**15 min**	Unlimited	Unlimited
Scaling	Per request, automatic	Service autoscaling	HPA / cluster autoscaler
Cold starts	Yes (mitigate with provisioned concurrency)	Task start (tens of seconds)	Pod start (seconds, if node available)
Long-lived streams (SSE/WebSocket)	Awkward	Fine	Fine
Sidecars / service mesh	No	Limited	**Yes**
Ops burden	Lowest	Low	Highest (but shared platform amortises it)
Best for	Spiky, short, event-driven tasks	Simple container services	Many services, mesh, platform standards

Why an agent runtime usually lands on Kubernetes: turns run for minutes and stream tokens (Lambda's 15-minute cap and API Gateway's ~29 s default integration timeout fight you), you want a mesh for mTLS, and a company-wide platform already exists.

4.4 Identity and secrets (IAM, STS, KMS)

- **IAM principal** — a user or a **role**. Services use roles, never long-lived keys.
- **Policy** — JSON saying which actions on which resources are allowed. **Least privilege**: only what's needed.
- **STS** — issues **temporary credentials** when something assumes a role.
- **IRSA / EKS Pod Identity** — maps a Kubernetes service account to an IAM role, so *each pod* gets only its own permissions (not the whole node's).
- **Secrets Manager / SSM Parameter Store** — store secrets, rotate them automatically.
- **KMS** — manages encryption keys. *Envelope encryption*: data is encrypted with a data key, and the data key is encrypted with a KMS key.

4.5 Data and messaging services

Service	What it is	Agent-platform use
S3	Object storage (files). Versioning, lifecycle rules, encryption (SSE-KMS), bucket policies	Knowledge corpus files, audit archives, eval artefacts
ElastiCache (Redis)	Managed in-memory store	Session/graph state, caches, idempotency keys, rate limits
RDS / Aurora	Managed relational DB (Postgres/MySQL)	Durable workflow state, audit tables
DynamoDB	Managed key-value at any scale	High-scale state, idempotency tables
SQS	Queue (each message to one consumer)	Background work, retries, DLQ
SNS / EventBridge	Pub/sub, event routing	Fan-out notifications
Kinesis / MSK (Kafka)	Ordered, replayable streams	Audit/feedback events, analytics
Bedrock	Managed access to foundation models (incl. Anthropic) inside AWS	In-region model calls, data stays in AWS
CloudWatch	Logs, metrics, alarms	(Many companies use Splunk/Grafana instead)

4.6 Resilience and DR

- **Multi-AZ** handles a data-centre failure; **multi-region** handles a regional failure.
- **RPO** (recovery point objective) = how much data you can lose. **RTO** (recovery time objective) = how long you can be down.
- DR strategies in increasing cost: backup/restore → pilot light → warm standby → active-active.

4.7 Infrastructure as Code (Terraform)

- You describe infrastructure in code (HCL): `resource "aws_s3_bucket" "corpus" {…}`.

- `terraform plan` shows the diff between code and reality; `terraform apply` makes it so.
- **State** records what Terraform created; teams keep it remote (e.g., S3 + DynamoDB lock) so two people can't apply at once.
- **Modules** are reusable building blocks (the platform team publishes them; service teams compose them).
- **Drift** = someone changed reality by hand; the next `plan` shows it.
- **Monitoring as code:** alerts and dashboards defined in Terraform too, reviewed like code.

4.8 In your system

Item	Value	Tag
Cloud	Hyperforce = Salesforce infrastructure on public cloud (AWS here)	[~]
Compute	Internal Kubernetes platform; services per instance/domain	[~]
Session store	Managed Redis (definition file not found)	[~] / [✗]
Knowledge corpus	S3 buckets → Data Cloud index	[~]
Models	LLM gateway routes to OpenAI, Azure OpenAI, Anthropic, Bedrock; **in-region Bedrock enforced**; kill switch to another region	[~]
Provider failover	OpenAI circuit breaker fails over to Azure	[~]
Region selection	Each org's AI region comes from its provisioned instance	[~]
Infra definitions	Terraform add-ons per service; **alerts/dashboards as Terraform**	[~] / [✓]
Accounts, IRSA roles, VPC layout, PrivateLink, RPO/RTO, cost	—	[✗]

4.9 Say it

> "Everything runs on Hyperforce — our infrastructure on AWS. The runtime services are containers on an internal Kubernetes platform, with session state in managed Redis, the knowledge corpus in S3 feeding a Data Cloud index, and model calls going through an LLM gateway that keeps Bedrock calls in-region and can fail over between providers. Infrastructure, alerts and dashboards are Terraform. The platform team owns accounts, networking and clusters; I'd describe my layer as the services and how they behave on that platform. If it helps, I can walk through why Kubernetes over Lambda for this workload."

4.10 Where he'll push

- **"Why not Lambda?"** 15-minute cap, cold starts on a latency-sensitive chat path, awkward streaming, no mesh/sidecars, and a shared Kubernetes platform already exists.
- **"How do pods get AWS permissions?"** Per-pod roles (IRSA/Pod Identity) with least privilege — *honest:* "on our platform that's platform-managed; I haven't configured the roles myself."

- **"A region goes down — what happens?"** Model calls: gateway kill switch routes to another region (residency trade-off — regulated orgs may not allow it). Knowledge: if the index lives in one region, every org's knowledge retrieval degrades — a single-region risk worth naming [~].
- **"How do you keep LLM traffic private?"** Bedrock via VPC endpoints/PrivateLink stays on the AWS network; external providers need egress through controlled gateways and contractual zero-retention.

> **Box — your hands-on AWS story (AdaptiveRAG on your resume).** If you built it yourself, this is where you can say "I configured…". Know: Lambda memory (128 MB–10 GB) also sets CPU; timeout (max 15 min); cold starts (and provisioned concurrency); the execution role's policy; zip vs container-image packaging; API Gateway's default ~29 s integration timeout (REST) and why long LLM calls push you to async or streaming; FAISS index loading at cold start (from S3 or the image). If you didn't build it hands-on, don't lead with it.

Chapter 5 — Data layer: Redis, the database, Data Cloud, event-driven

5.1 Redis from scratch

What it is. An in-memory key-value store. Microsecond reads, rich data types:

Type	Example use
String	Cached value, counter, idempotency marker
Hash	A session object with fields
List / Stream	Queues, event logs
Set / Sorted set	Unique members; leaderboards; time-ordered items

TTL. Any key can expire (`SET key value EX 3600`). Sessions, caches and idempotency markers almost always have TTLs.

Memory full → eviction policy.

Policy	Behaviour
`noeviction`	Writes fail when full
`allkeys-lru`	Evict least-recently-used keys
`volatile-lru`	Evict LRU among keys *with* a TTL
`volatile-ttl`	Evict keys closest to expiring

Durability. Redis is memory-first. **RDB** = periodic snapshots; **AOF** = append-only log of writes. Managed Redis often runs with replicas rather than strong persistence. A failover can lose the last moments of writes.

High availability. Primary + replicas; automatic failover promotes a replica. **Cluster mode** shards keys across nodes by hash slot (16,384 slots).

Common patterns.

- **Session store** — keep conversation state between HTTP turns.
- **Cache-aside** — read cache → on miss read DB → write cache with TTL.
- **Idempotency** — `SET dedupe:<key> 1 NX EX 86400` : only the first request "wins".
- **Rate limiting** — counters with TTL per tenant.
- **Locks** — `SET lock NX PX` (with caveats: a lock isn't safe if the holder pauses past expiry; use fencing tokens for correctness).

The golden rule: Redis must not be the only home of anything you can't afford to lose or re-derive. If approval state lives only in Redis and the key is evicted, the approval is gone.

5.2 Relational databases from scratch

- **Transaction (ACID):** a group of writes that all happen or none do (Atomic), keep rules intact (Consistent), don't see each other's half-done work (Isolated), and survive crashes (Durable).
- **Isolation levels** (weakest → strongest): read committed → repeatable read / snapshot → serializable. Weaker levels allow anomalies:
- **Lost update** — two transactions read-modify-write the same row; one overwrite wins silently.
- **Write skew** — two transactions each check a condition, both pass, together they break it.
- **Locking strategies:** *pessimistic* (lock the row, `SELECT … FOR UPDATE`) vs *optimistic* (version column; `UPDATE … WHERE version = 7` fails if someone else changed it — compare-and-set).
- **Multi-tenancy in one database:** every row carries a tenant ID; every query filters by it; indexes lead with it.

Salesforce specifics you should know:

- Every row is scoped by **org ID**; code reads the org from the user context.
- **Governor limits** cap DML statements, queries and CPU per transaction.
- `UNABLE_TO_LOCK_ROW` — row-lock contention error when two transactions update related records (common with group membership and sharing recalculation).
- DML can be **all-or-none** or **partial success** (per-record results).

Durable workflow state (your ResolveFlow, hands-on): LangGraph *interrupts* pause a graph for human approval; a **Postgres checkpointer** persists the state at each step, so the run survives restarts and resumes exactly where it paused. This is the durable alternative to keeping approval state only in a cache.

5.3 Data Cloud and the retrieval path (from scratch)

```
source files (S3) → data stream → DLO (raw data lake object)
   → DMO (harmonised data model object) → chunking → embeddings
   → vector + keyword (hybrid) index → retriever (top-k, threshold, rerank)
```

- **Chunking** splits documents into passages; **embedding** turns each into a vector.
- **Hybrid search** combines keyword (exact terms) and vector (meaning) scores — *fusion*.
- **Freshness lag** = time from a source change to it being searchable. Every stage adds lag.

5.4 Event-driven design from scratch

Queue vs stream.

- **Queue (SQS):** each message goes to one consumer, then disappears. Work distribution.
- **Stream / log (Kafka, Kinesis):** messages are kept in order in **partitions**; many **consumer groups** read independently, each tracking its **offset**; you can replay.

Delivery semantics.

Semantics	Meaning	Reality
At-most-once	May lose messages, never duplicates	Fire-and-forget
At-least-once	Never loses, may duplicate	**The default almost everywhere**
Exactly-once	Neither	Only *effectively* — at-least-once + **idempotent consumers**

Supporting patterns.

- **Dead-letter queue (DLQ):** messages that keep failing go aside for inspection instead of blocking the queue.
- **Ordering:** guaranteed only within a partition — pick a partition key (e.g., tenant or user ID) for events that must stay in order.
- **Transactional outbox:** write the business row *and* an "event to publish" row in the same DB transaction; a relay publishes it. Avoids "DB updated but event lost" (the dual-write problem).
- **Backpressure:** when consumers fall behind, queues grow — alert on queue depth/age, scale consumers on it.

Salesforce: *Platform Events* are Salesforce's pub/sub bus inside the platform.

5.5 In your system

Item	Value	Tag
Agent API session	Stored in **Redis** by a session-cache service; Redis is that service's only external data dependency	[✓]
Graph approval flags	`todo_approved` / `current_plan_shown` are **graph state variables**; separate Redis key, TTL, eviction behaviour	[✓] / [✗]
What gets written	`Group` row (create action) and `GroupMember` rows (membership action, draft then commit) in the Core DB	[✓]
Tenant	Org ID taken from the user context in Core	[✓]
Transaction across both actions	**None** — two separate commits	[✓]
Membership save modes	All-or-nothing or standard (per-record results)	[~]
Streaming to the UI	Server-sent events (SSE), long-lived connection (~10-minute cap)	[~]
Events	Planner action failures published as events; LLM gateway feedback/audit on Kafka; deferred audit to S3	[~]
Knowledge ingestion	**Manual**: S3 upload + index rebuild on several orgs + Core jar bump (12–24 h)	[~]
Session analytics	Agent session/interaction records land in Data Cloud objects	[~]

5.6 Say it

> "State lives in three places. Conversation state between turns is in Redis behind the Agent API — fast, with TTLs. The graph's own state, including whether the plan was shown and approved, is graph state the reasoner updates. And the real writes — the group and its members — are rows in the Core database, scoped by org ID from the user's context. Knowledge is a separate path: skill documents go from S3 into a Data Cloud hybrid index that retrieval queries. Events like action failures and gateway audit go out asynchronously over Kafka. The rule I hold myself to is that Redis is never the only copy of anything we can't re-derive — approval decisions especially."

5.7 Where he'll push

- **"Redis fails over while an admin is looking at the plan — what happens?"** If the session or graph state is lost, the next message starts fresh and the old approval can't be applied. Fix: persist plan + approval durably (DB/checkpointer) keyed by plan ID; Redis as a cache in front. *Honest:* key and TTL are [✗].
- **"Two admins approve conflicting plans for the same group?"** Today no lock was found [✗]. Design: optimistic concurrency — record the group's version at plan time, re-validate at execute time, fail closed with "this changed since you approved".
- **"Double-click on Approve?"** Idempotency key = hash(plan ID + step + version); first execution records it; second is a no-op returning the first result.
- **"At-least-once events — how do consumers cope?"** Idempotent consumers with a processed-ID table/marker; DLQ after N attempts; alert on DLQ depth.

Chapter 6 — CI/CD

6.1 From scratch: the vocabulary

- **CI (Continuous Integration):** everyone merges small changes often; every change is automatically built and tested. Goal: find breakage within minutes of causing it.
- **Continuous Delivery:** every change that passes the pipeline is *releasable* — deploying is a button.
- **Continuous Deployment:** every passing change goes to production automatically.
- **Pipeline:** an ordered set of **stages** (build → test → scan → package → deploy) made of **jobs**, with **gates** (conditions that must pass) between them.
- **Artifact:** the output that gets deployed — a container image, a jar.
- **Environments:** dev → staging/perf → production, with **promotion criteria** between them.

6.2 A typical pipeline, stage by stage

```
commit/PR ─► pre-commit / pre-push hooks (lint, fast checks)
          ─► CI on the PR: build · unit tests · static analysis · SAST · dependency scan
          ─► code review + required checks ─► merge
          ─► post-merge: integration/functional tests · coverage · build image · image scan · push to
registry
          ─► deploy to staging ─► smoke tests / functional tests / (evals)
          ─► progressive production rollout (canary → analysis → expand)
          ─► post-deploy verification ─► (auto) rollback on failure
```

6.3 Deployment strategies

Strategy	How	Pros	Cons
Recreate	Stop old, start new	Simple	Downtime
Rolling	Replace pods a few at a time	No downtime, default in K8s	Old and new run together; slow rollback
Blue/green	Stand up the full new version ("green") beside old ("blue"); switch traffic at once	Instant switch-back	Double capacity during switch
Canary	Send a small % of traffic to the new version, watch metrics, expand	Limits blast radius; data-driven	Needs good metrics; slower
Shadow	Copy real traffic to the new version, discard its responses	Zero user risk	Can't shadow writes safely

Progressive delivery = canary + automated analysis: at each step, query error rate/latency; if worse than baseline beyond a threshold → automatic rollback.

Staggered / wave rollouts in multi-instance platforms: deploy to one instance or domain first, **soak** (wait and watch), then waves of more instances.

6.4 Feature flags: deploying ≠ releasing

A **feature flag/gate** lets code reach production *switched off*, then be turned on per tenant, per user, or per percentage. Benefits:

- Decouple deploy (engineering risk) from release (business decision).

- **Kill switch** — turn a feature off in seconds without a deploy.
- Per-tenant rollout and per-tenant rollback.

Cost: flag debt — old flags must be removed.

6.5 Rollback, roll-forward, and the compatibility rule

- **Rollback** = return to the previous artifact. **Roll-forward** = ship a fix. Rollback is safer *if* everything you changed can be reverted together.
- **The compatibility rule (expand/contract):** when two things ship separately (code and DB schema; or graph and actions), make every change backward compatible: 1. **Expand** — add the new field/action; old and new code both work. 2. Migrate callers. 3. **Contract** — remove the old one only when nothing uses it.

6.6 Release trains, freezes, config drift

- **Release train:** a big system ships on a fixed schedule; your change rides the next train (Salesforce Core ships in major releases plus patches).
- **Freeze windows:** no deploys around high-risk events (e.g., a flagship conference).
- **Config drift:** configuration applied by hand diverges from what's in Git. Fix: config applied only by the pipeline (GitOps).

6.7 CI/CD for agents (what's different)

An agent changes through more than code: **prompts, skill documents, tool schemas, the graph, the model version**. Each is a deployable artefact that can regress behaviour. So:

- Version them all and record which versions each run used.
- Run **evals** as the regression tests: deterministic checks (right tool, right arguments, permission check fired) can gate; LLM-judged quality is noisier and starts as a warning.
- A model upgrade is a release — canary it like code.

6.8 In your system — two lanes

```
LANE 1 — Core artefacts (graph YAML, skill docs, actions)
  PR → pre-push hook blocks graph changes that violate invariants I-1..I-6      [✓]
     → pre-merge CI (precheckin pipeline)                                       [✓]
     → post-commit: unit, functional tests, coverage                            [✓]
     → Core build (Bazel) → Core release train (patch branch)                   [✓]
     → features turned on per org by gates                                      [✓]
  Graph only: weekly promotion — validated graph becomes live, previous becomes
  "Stable"; a rotating owner runs it                                            [~]
  Rollback: per-org gate switches that org to the Stable graph                  [✓]
    → reverts ONLY the graph, not actions/schemas/skill docs                    [✓ code + ~ Slack]
    → in source, the gate is open for one test org only                         [✓]

LANE 2 — Runtime services (Agent API, worker, planner, reasoner, gateways)
  PR → internal CI builds a Docker image → registry                             [~]
     → Helm + Terraform deploy                                                  [~]
     → staged rollout across instances/domains: canary, blue/green, soak        [~]
     → health analysis → rollback (automatic on the LLM gateway; 1-min pause)   [~]
     → functional integration tests gate some deploys                           [~]
  One service's platform config applied by hand, outside CI (drift risk)        [✓]

KNOWLEDGE — skill doc corpus
  separate jar → manual S3 upload + index rebuild → Core jar bump (12–24 h)     [~]

EVALS — harness and gold suite run, but do NOT gate any release                 [~]
```

Not found [✗]: pipeline files, stage durations, approvers, canary percentages, bake times, who can roll back and how long it takes.

6.9 Say it

> "There are two lanes. The agent's definition — graph, skill documents, actions — lives in Core, so it goes through Core CI: a pre-push hook that blocks graph changes violating our invariants, pre-merge checks, post-commit functional tests, then the Core release train, and features are turned on per org with gates. Graph changes are promoted weekly, and any org can be dropped back to the stable graph with a gate. The runtime services each build a container image and roll out through Helm and Terraform in stages across instances, with health analysis and rollback. The weakness I'd call out is rollback scope: the gate reverts only the graph, not the actions or skills, so we keep those backward compatible — expand, then contract. And evals today are advisory; I'd gate the weekly promotion on deterministic eval checks."

6.10 Where he'll push

- **"How long does a rollback take and who does it?"** [✗] — "I don't have the number; the graph rollback is a gate flip, so seconds-to-minutes once someone with access acts; service rollbacks go through the deploy tooling."
- **"You changed a tool's input schema; a plan approved yesterday executes today. What happens?"** It may send old-shape arguments. Fix: schema versioning, accept both shapes during transition (expand/contract), re-validate the plan at execution and ask for re-approval if it no longer validates.
- **"How do you safely roll out a model upgrade?"** Offline evals vs the current model → canary to a small set of orgs/traffic → compare online metrics (tool accuracy, grounding failures, latency, cost) → expand; keep the old model behind a flag for instant fallback.

- **"Why weekly and not continuous?"** Graph changes alter behaviour for every org; a weekly cadence gives time to run full eval suites and have an owner accountable. With an automated eval gate you could go faster.

> **Box — AutoRABIT (your CI/CD-for-Salesforce background).** Salesforce DevOps differs from container CI/CD: you deploy **metadata** (objects, Apex, flows, permission sets) through the **Metadata API** into **orgs**. Know: sandboxes and scratch orgs; source-tracked (SFDX) format in Git; **validation-only deploys** (check-only) and **quick deploy** of a validated package; production deploys require **≥ 75% Apex test coverage**; destructive changes; ordering/dependency failures as the classic cause of failed deployments. "Release reliability ~30%" — know whether that's deployment success rate, rollback rate, or failed-deployment rate, and over what period.

Part 2 — Seeing, controlling and securing it

Chapter 7 — Observability

7.1 From scratch: monitoring vs observability

- **Monitoring** answers questions you knew to ask in advance ("is error rate above 1%?").
- **Observability** lets you answer questions you *didn't* anticipate ("why did only this tenant's turns get slow after 3 p.m.?") from the data the system already emits.

The raw material is three kinds of signal, tied together by **correlation IDs**.

7.2 Logs

- A **log** is a record of one event. **Structured logs** are key-value (usually JSON), so you can filter and aggregate: `{"ts":…, "level":"ERROR", "trace_id":"…", "org_id":"…", "action":"CreateGroup", "status":"FAILED", "latency_ms":5123}`.
- **Levels:** DEBUG / INFO / WARN / ERROR. Production usually keeps INFO+.
- **Fields that should always be present:** timestamp, service, trace/correlation ID, tenant (org) ID, user ID (or a hash), operation, status, latency.
- **Never log:** secrets, tokens, raw PII. Redact at the source.

Splunk basics. Logs are stored in **indexes** with a **sourcetype**. You query with **SPL**:

```
index=<app_index> sourcetype=<service> action=CreateGroup status=FAILED earliest=-1h
| stats count by org_id, error_code
| sort -count
```

```
index=<app_index> trace_id=<id>
| sort _time
| table _time service step status latency_ms
```

```
index=<app_index> sourcetype=<llm_timing> earliest=-24h
| timechart span=15m perc95(duration_ms) by template
```

(Field names here are illustrative — use the shape, not the exact names.)

7.3 Metrics

- A **metric** is a number over time, cheap to store and fast to alert on.
- **Types:** *counter* (only goes up: requests served), *gauge* (goes up and down: queue depth), *histogram* (distribution: latency buckets → percentiles).
- **Percentiles:** p50 (median), p95, p99. Averages hide the tail — always alert on percentiles.
- **Cardinality:** each unique label combination is a separate time series. Never put user ID or request ID as a metric label — that belongs in logs and traces.

Frameworks for "what to measure":

Framework	For	Signals
Golden signals (Google SRE)	Any service	Latency, traffic, errors, saturation
RED	Request-driven services	Rate, Errors, Duration

Framework	For	Signals
USE	Resources (CPU, memory, pools)	Utilisation, Saturation, Errors

7.4 Distributed tracing

A user request crosses many services. A **trace** is the whole journey; it's a tree of **spans**. Each span = one operation (an HTTP call, an LLM call, a DB query) with a start, a duration, attributes, and a parent.

```
trace 4bf92f…  (turn: "create group Sales Ops")
├─ span: Agent API  /messages                1.2 s
│  └─ span: worker → planner                 1.1 s
│     └─ span: reasoner: Knowledge node      0.9 s
│        ├─ span: retrieval (AI gateway)     0.3 s
│        └─ span: LLM call (gateway)         0.5 s
```

Context propagation. For spans in different services to join one trace, the trace ID travels with the request. The W3C standard header:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             ver  trace-id (16 bytes)           parent span     sampled
```

Where traces break (memorise — this is where systems lose visibility):

- **Async hops** — a message on a queue doesn't carry the header unless you put it in the message.
- **Long pauses** — a human approval resumes as a *new* HTTP request with a new trace.
- **Streams** — SSE/WebSocket connections outlive one request.
- **Third-party hops** — Slack, external MCP clients.

The fix is a **business correlation ID** (session ID + plan ID) that you stamp on every log line and span, independent of the trace ID.

OpenTelemetry (OTel). The vendor-neutral standard: SDKs in each service create spans/metrics/logs; a **collector** receives, processes and exports them to any backend (Splunk, Grafana, New Relic, …).

Sampling.

- **Head-based:** decide at the start of a request ("keep 1%"). Cheap, but rare failures are mostly thrown away.
- **Tail-based:** buffer the whole trace, then decide — keep all errors and slow traces, sample the rest. Better for debugging; needs a collector tier.

7.5 SLIs, SLOs, error budgets, burn rate

- **SLI** — a measurement of user experience: "fraction of turns that complete without error", "p95 time to first token".
- **SLO** — the target: "99.5% of turns succeed over 30 days".
- **SLA** — the contractual promise (with penalties), usually looser than the SLO.
- **Error budget** — what the SLO allows to fail. 99.9% over 30 days = **43.2 minutes** of full outage.
- **Burn rate** — how fast you're spending the budget. Burn rate 1 = you'll use exactly the budget in 30 days.

- **Multi-window burn-rate alerts** (Google SRE workbook): page if burn rate > **14.4** over both 1 h and 5 min (2% of budget gone in an hour); page if > **6** over 6 h and 30 min; ticket if > **1** over 3 days. Two windows = fast detection *and* fast reset.

7.6 Alerts and dashboards

- **Alert on symptoms users feel** (errors, latency, failed turns), not on every cause (CPU high).
- Every page must be **actionable** and link a **runbook**.
- **Severity and routing:** Sev1/Sev2 → pager (PagerDuty); lower → Slack/ticket.
- **Dashboards:** the top row is the golden signals for the user journey; below that, per-dependency panels (LLM gateway, retrieval, Core, Redis).

7.7 Observability for agents (what's different)

Uptime isn't enough — an agent can be *up* and *wrong*. Instrument:

What	Why
A span per **LLM call**: model, prompt/template version, tokens in/out, latency, finish reason	Cost, latency, regressions after a model/prompt change
A span per **tool call**: name, argument hash, status, latency	Tool failure rates; which tool broke
A span per **retrieval**: k, top scores, document IDs	Empty/weak retrieval → hallucination risk
Guard events: validation result, approval shown/approved/rejected, confirmation	Safety funnel; approval latency and abandonment
Grounding check result per response	Correctness signal in production
Cost per turn and per tenant	Noisy neighbours, budgets
Online evals: score a sample of production traces with the offline scorers	Catch quality drift without a user complaint

Karan's own vocabulary (his New Relic post): propagate `traceId`, `upstreamApplication`, `tenantId` and `userId` as attributes on every hop. Use those words.

7.8 On-call and incidents

Incident lifecycle:

1. **Detect** — alert or user report.
2. **Triage** — what's the blast radius? which tenants? since when? what changed (deploy, flag, model, dependency)?
3. **Mitigate first** — roll back, flip a flag, fail over, shed load. Restore service before root-causing.
4. **Communicate** — status updates on a cadence.
5. **Resolve** and verify.
6. **Blameless postmortem** — timeline, root cause, contributing factors, action items with owners; add a **regression test/eval case**.

First 15 minutes of a page (say this shape): acknowledge → check the dashboard's top row → check recent changes → scope by tenant/region → mitigate → post an update.

7.9 In your system

Signal	What exists	Tag
Core log: LLM timing	Record with trace ID, template name, generation ID, durations, token counts, error message	[~]
Core log: action	Record with action, status, input, output, time (ms), target group ID	[~]
Runtime logs	Separate Splunk indexes for the agent runtime, the planner, and the retriever	[~]
Planner trace	Typed steps per plan: user input, LLM step (prompt, response, latency), function step (name, input, output), transition, response type, **grounding check (GROUNDED/UNGROUNDED)**; keyed by plan ID + session ID	[~]
Planner → Core request tags	planner type, session ID, turn ID, plan ID, user context, agent version	[~]
Trace IDs	Zipkin-style trace ID in Core; OTel sampled at the LLM gateway	[~] (not re-verified)
Alerts/dashboards	Defined **as code** (Terraform) for the Agent API, worker, planner, reasoner and others; Grafana boards	[✓] README
Retriever alert	Availability Sev2 < 99.7%, Sev1 < 99.3% (60-min window, 10-min persistence) → **Slack only; paging off**	[~]
SLOs	**None found**	[✗]
One correlation ID across every hop and the approval pause	**Not found**	[✗]
Awaiting-approval metric (pause length, abandonment)	**Not measured**	[~]
Harness log lookup	Script searches **preprod** only	[~]
A real incident	Data-query action 500s; egress-mesh call to an API-protection dependency failed to create request context; 3 retries → 500; generic apology shown to users; self-resolved in hours	[~] digest

7.10 Debugging walkthrough: "the agent said something went wrong"

1. Get the **session ID** (and time) from the user or UI.
2. Core logs: LLM-timing records for that session → which LLM call failed or was slow?
3. Planner trace → which **step** failed: LLM, function (tool), or grounding?
4. Action record → status, latency, error code.
5. If grounding was empty → retriever logs and gateway dashboard (timeouts, 429s).
6. Compare the turn's duration with the **timeout chain** — did an outer layer give up while the inner one succeeded?
7. Check **what changed**: deploy, gate, graph promotion, model.

7.11 Say it

> "Every layer writes structured records to Splunk. In Core, LLM calls log trace ID, template, durations and token counts, and actions log status, latency and the target record. The planner emits a typed trace of every step — LLM calls, tool calls, transitions — including a grounding check on each response. Alerts and dashboards are code in Terraform. The gap I'd fix first is correlation: one user request with an approval is several HTTP turns, through a stream and a worker, so a query on the first trace ID misses the execution. I'd stamp session ID plus plan ID, with org ID and user ID, on every hop — the traceId, tenantId, userId pattern — so 'where is request X' becomes one query. And I'd add SLOs: turn success rate and time-to-first-token p95 under three seconds, with burn-rate alerts."

7.12 Where he'll push

- **"Head or tail sampling for agents?"** Tail — failures and slow turns are rare and exactly what you need. Head at 1% drops ~99% of failures.
- **"What would page someone at 3 a.m.?"** Turn success rate burn-rate breach; time-to-first-token p95 breach; action failure spike; LLM gateway/retrieval availability. *Not* CPU.
- **"How do you detect the agent getting *worse* without errors?"** Grounding-failure rate, tool-selection accuracy from online evals on sampled traces, clarification and rejection rates, user re-asks.
- **"Cardinality — can I tag metrics by org?"** Org is bounded but large (thousands); OK for some metrics with care, otherwise top-N + logs. User ID: never on metrics.

Chapter 8 — The agent's tools (actions, discovery, MCP)

8.1 From scratch: what a "tool" is to an LLM

An LLM only produces text. **Tool use (function calling)** lets it request an action in a structured way:

1. You give the model a list of tools — **name, description, input schema (JSON Schema)**.
2. The model replies with a tool call: `{"name":"create_group","arguments":{"label":"Sales Ops","type":"Regular"}}`.
3. **Your code** validates the arguments, decides whether it's allowed, executes it, and returns the result.
4. The model continues with the result.

The model never executes anything. Your application does — which is where safety lives.

8.2 The tool lifecycle (design principles)

Stage	Principle
Define	Typed contract: input and output schemas; clear, specific descriptions (the model chooses by description).
Register / allowlist	Only registered tools can ever be offered; each has an owning team.
Discover / select	With hundreds of tools, don't show all of them — **retrieve** the few relevant ones (semantic search over tool descriptions) with a **similarity threshold** to keep weak matches out.
Scope by state	Offer only tools valid for the current step ("executor may only call write tools while a task is running").
Gate	Validate arguments against the schema; require approval/confirmation for writes.
Execute	Under the **user's** identity and permissions, re-checked in code (Ch 10). Idempotently.
Observe	Log tool, status, latency, and outcome; eval tool selection offline.

8.3 Human-in-the-loop patterns

- **Plan approval** — show the whole plan; nothing executes until approved.
- **Per-tool confirmation** — a specific write pauses for "confirm?".
- **Draft-then-commit (dry run)** — compute the change without saving, show it, then commit.
- **Design rules:** approval must be *bound* to exactly what will execute (plan ID + version); re-validate at execution (state may have changed); approvals expire; safety flags have **one source of truth** and **fail closed**.

8.4 MCP from scratch

Model Context Protocol = an open standard (from Anthropic, now broadly adopted) for connecting AI applications to tools and data, so each integration is built once instead of once per app.

Roles.

- **Host** — the AI application (Claude Desktop, an IDE, an agent runtime).
- **Client** — a connector inside the host; one client per server.
- **Server** — exposes capabilities: **tools** (actions), **resources** (readable data), **prompts** (templates).

Wire format. JSON-RPC 2.0 messages.

Lifecycle.

1. `initialize` — client and server agree on protocol version and capabilities.
2. `notifications/initialized`.
3. Operations: `tools/list`, `tools/call`, `resources/read`, `prompts/get`, …
4. Shutdown.

Transports.

- **stdio** — the host launches the server as a local subprocess.
- **Streamable HTTP** — a remote server over HTTP (can stream with SSE); the current remote transport.

Tool definition. `name`, `description`, `inputSchema` (JSON Schema), optional `outputSchema`, and **annotations** hints: `readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`.

Auth for remote servers. OAuth 2.1: the MCP server is a resource server; clients obtain tokens for that specific server (audience-bound). **Token passthrough is forbidden** — a server must not forward the token it received to other APIs (that's the confused-deputy trap).

Security risks to name. Tool poisoning (malicious descriptions), prompt injection via tool output, over-broad tools, confused deputy, excessive permissions.

Coarse vs fine-grained tools.

	Fine-grained (one MCP tool per action)	**Coarse (one tool = "talk to the agent")**
Caller control	High	Low
Safety	Caller can skip your approval flow unless enforced server-side	Your agent's approvals and checks always run
Latency	Low per call	High (a full agent turn)
Composability	High	Lower

8.5 In your system

Item	**Value**	**Tag**
Tool definition	Action YAML + input/output **JSON schema** + Java invocable action in Core	[✓]
Allowlist	Agent action allowlist with an owning team per entry	[~]
Discovery	System primitives: semantic search, ~50 hits, keep **similarity ≥ 0.7**, then **top 2**; global vector fallback: up to **5 candidates per goal**	[✓]

Item	Value	Tag
State scoping	Executor may call the create-group tool only while task status is *running* and the tool is in the state's **enabled tools**	[✓]
Plan approval	Plan shown → explicit approve; **only single-task plans auto-approve**	[✓]
Per-tool confirmation	Graph says *confirm* for create-group; the action definition says *don't*; which wins depends on the compile path	[✓] (open question)
Draft-then-commit	Membership: dry run (`commitChanges=false`) then commit	[✓]
Execution	mTLS from the planner into Core; action runs its own access check	[✓]
MCP	The Setup agent exposed as an MCP server with **one coarse tool** ("chat with setup agent"); a proxy speaks MCP outward and the Agent API inward; session key = (org, MCP session, bot)	[✓]
MCP latency (local sample)	p50 ~35 s, p95 ~98 s, p99 ~117 s; proxy timeout 60 s × 2 retries; 3-min ceiling	[✓] spec, labelled directional
"Eight capabilities onboarded to Headless 360"	—	[✗]

8.6 Say it

> "Every tool is a typed contract — a definition with JSON schemas for input and output, backed by Java code in Core, and allowlisted. With hundreds of capabilities you can't show the model everything, so tools are retrieved: semantic search with a 0.7 similarity floor, keeping the top two, with a broader fallback. The executor only sees tools enabled for its current state, and writes go through three human-in-the-loop layers — plan approval, per-tool confirmation, and draft-then-commit for membership. For MCP we deliberately exposed the whole agent as one coarse tool, so any external agent gets a conversation, never raw write primitives — our approval and access checks can't be bypassed. The cost is latency: a full agent turn per call."

8.7 Where he'll push

- **"Why one coarse MCP tool?"** Safety over flexibility. Revisit by exposing read-only tools fine-grained (annotated `readOnlyHint`) and keeping writes behind the agent.
- **"How does the MCP server know which user and tenant?"** Session key includes org; caller context is passed into the agent — *honest:* the exact header carrying the user was [✗]. Correct design: an audience-bound OAuth token for the user, exchanged (not passed through) for downstream calls.
- **"Similarity 0.7 — how did you choose it?"** Tune on a labelled set: plot precision/recall of the correct tool appearing vs threshold; pick the knee; re-tune when the tool catalogue changes. *Honest if you didn't tune it.*
- **"Two places define the confirmation flag — so what?"** A safety setting with two sources of truth means behaviour depends on code path. Rule: one source, fail closed (if any says confirm, confirm), plus a test asserting the effective value.

Chapter 9 — Engineering tooling: evals, harnesses, triage

9.1 From scratch: why agents need evals

Unit tests check deterministic code. An agent's behaviour depends on a model, prompts, retrieved documents and tool descriptions — all of which change. **Evals** are the regression tests for behaviour.

Two kinds of checks:

Deterministic checks	Judged checks
Did it call the right tool? With the right arguments? Did the permission check fire? Did it ask for approval before writing? Is the output valid JSON?	Is the answer helpful? Grounded in the retrieved docs? Polite? Correct explanation?
Cheap, stable, **can gate releases**	Need a human or an **LLM judge**; noisier

Golden set — a curated, versioned set of inputs with expected behaviour. Mix: normal, ambiguous, adversarial, failure-path.

Replaying production failures — every real failure becomes a new eval case (input + expected behaviour), so it can't silently regress.

Non-determinism — run each case several times; report **pass rate** and **stability** (same result across runs).

9.2 LLM-as-judge and Cohen's kappa

An LLM judge scores outputs against a rubric. You must check it agrees with humans.

Cohen's kappa (κ) measures agreement between two raters *beyond chance*:

```
κ = (p_o − p_e) / (1 − p_e)
p_o = observed agreement      p_e = agreement expected by chance
```

Example: judge and human agree on 85% of cases; by chance they'd agree on 55% → κ = (0.85 − 0.55)/(1 − 0.55) = **0.67**.

κ	Common reading (Landis & Koch)
0.41–0.60	Moderate
0.61–0.80	**Substantial**
0.81–1.00	Almost perfect

Your judge audit was **κ ≈ 0.65** [~] — low end of substantial: good enough to *inform*, not to *block* releases alone.

9.3 Gating strategy (what you'd propose)

1. **Gate** the weekly graph promotion on **no regression in deterministic checks** against the stable set (tool, arguments, permission check, approval before write).
2. **Warn** (don't block) on judged dimensions until κ > ~0.7 on a fresh audit.
3. **Block on safety cases at 100%** — an unapproved write is never acceptable.
4. Run a **smaller smoke set per PR**, the full suite nightly and before promotion.
5. **Online:** score sampled production traces with the same scorers.

9.4 In your system

Item	Value	Tag
Harness	Command-line tool built on an MCP harness; drives utterance suites against preprod/dev orgs; results **PASS / FAIL / SOFT_FAIL**	[~]
Scoring engine	A separate scoring tool	[~]
Suites	Quarterly utterance suite ~**524** utterances — latest five-run report **~82% pass, ~88% stable**; daily backend regression **499 rows across 53 actions** (partly Haiku-judged); vision-graded UI runs	[~]
Gold suite	33 API cases + 15 UI fixtures — **recommended, not required**	[~]
Judge audits	Kappa audit folders (Aug 2026); **κ ≈ 0.65**	[✓] folders / [~] value
CI gate on pass rate	**None**	[✗]
Pre-push hook	Blocks graph changes that break invariants	[✓]
Production failure → eval case path	Not written down anywhere found	[✗]

9.5 AI Bug Triage skill (your hackathon project)

A Claude Code workflow that, given a bug, **collects the evidence a human would**: Splunk logs for the failing request, the relevant repository code and recent changes, and the work item — then produces a structured investigation report (evidence, likely failure paths, root-cause hypotheses). **Diagnostic-context collection: ~90 → ~15 min.** Be ready to say *how* you measured that (e.g., timed on N real bugs before/after).

9.6 Say it

> "We evaluate behaviour, not just code. A harness drives utterance suites against test orgs and grades each case pass, fail or soft-fail. Deterministic checks cover the right tool, the right arguments, whether the permission check fired and whether approval came before a write. An LLM judge covers quality, and we audit it against human labels — kappa about 0.65, the low end of substantial, so it informs but doesn't block. We run cases several times because the model isn't deterministic: the latest report was about 82% pass and 88% stable. Today it's advisory. What I'd do is gate the weekly promotion on zero regressions in the deterministic and safety checks, keep judged scores as warnings until agreement improves, and score sampled production traces with the same scorers."

9.7 Where he'll push

- **"82% pass — is that good?"** Depends on the mix (ambiguous/adversarial cases fail by design); what matters is no regression vs the stable baseline and 100% on safety cases.
- **"How do you stop eval sets from going stale?"** Feed production failures and new capabilities in continuously; retire duplicates; version the set.
- **"How do you know the judge isn't biased toward its own model's style?"** Use a different model family as judge, audit with κ, blind position/ordering, calibrate with anchors.

Chapter 10 — Identity and security across the stack (his home turf)

10.1 TLS and mTLS from scratch

- **TLS** encrypts a connection and lets the **client verify the server** via the server's certificate (signed by a Certificate Authority the client trusts).
- **mTLS (mutual TLS)** adds the reverse: the **server verifies the client's certificate** too. Both sides prove who they are.
- In a mesh, every service gets a short-lived certificate encoding its identity (e.g., a SPIFFE ID like `spiffe://domain/ns/x/sa/planner`); certificates rotate automatically.
- **mTLS proves which *service* is calling. It says nothing about which *user* the call is for.**

10.2 The two-identity problem

Every write an agent makes on behalf of a person involves:

1. **Workload identity** — "this request comes from the planner service" (mTLS).
2. **User identity** — "…acting for admin Priya, whose permissions apply".

How user identity can travel (weakest → strongest):

Method	Risk
A header with a user ID, trusted because the caller is mesh-authenticated	**Confused deputy:** any compromised trusted service can claim to be any user
The user's own token passed through	Token replay; wrong audience; forbidden by MCP
Token exchange / On-Behalf-Of — trade the user's token for a new, audience-bound token that names both the user (`sub`) and the acting service (`act`)	Best: scoped, short-lived, auditable

OAuth 2.0 Token Exchange (RFC 8693) — the standard for that last row. **Azure AD's On-Behalf-Of flow** is the same idea (Karan spent 5½ years in Azure AD — say "on-behalf-of").

10.3 Principles to say out loud

- **Authorization is decided in code, never by the model.** The model chooses; the code permits.
- **Least privilege:** the agent can never do more than the user could do themselves.
- **Re-check at execution time** (permissions can change between plan and execute; approvals can be hours old).
- **Fail closed:** misconfiguration must deny, not degrade.
- **Audit:** who asked, who approved, what exactly changed, under whose authority, when.
- **Secrets:** short-lived, rotated, never in logs or Git.

10.4 In your system

Item	Value	Tag
Service → Core in production	**mTLS** (Core port 9443); certificate refreshed in place	[✓]
Legacy token path	Only runs in local/dev environments	[✓]
Misconfiguration	Production raises an error if the mTLS client is missing — **fails fast, never silently degrades**	[✓]
Local-only identity headers	Hard-coded service identity headers injected only on the local workspace port	[✓]
User attribution	A separate header (not the certificate) — **which user ID Core sees is [✗]**	[✗]
Action authorization	Invocable action's own current-user access check + the skill's declared permissions (org feature + View Setup)	[✓]
Model's reach	Only tools enabled in the current state; executor only while a task runs	[✓]
ResolveFlow (your repo)	**GitHub OAuth** — approved writes run under the authenticated user's own permissions; write functions refuse without valid authorization	Your hands-on

10.5 Say it

> "There are two identities on every write. The workload identity is mTLS — the certificate proves the call came from our planner service, and production has no fallback: if the certificate client is missing, it fails fast instead of dropping to a weaker token path. The user identity travels separately as request context, and Core rebuilds the user context from it, so the action's access check and the permissions the skill declares are evaluated for the admin who asked. The model never makes the authorization decision — it can only pick tools enabled for the current state, and the action re-checks in code. Where I'd push further is binding the user cryptographically: today the user context rides on a mesh-trusted call, which is a confused-deputy risk; an on-behalf-of token exchange, where the downstream token carries both the user and the acting service, closes it."

⚠ **Before Thursday:** confirm which user ID Core actually receives on the action call (office-Claude prompt in GITLAB_ANSWERS.md §7). If it's an integration user, change the resume line and the answer above.

10.6 Where he'll push

- **"What's a confused deputy?"** A trusted component tricked into using its authority for someone who shouldn't have it.
- **"User loses a permission after approving but before execution — what happens?"** Correct design: the action's access check runs at execution time against the current user context, so it fails closed. *Verify* our re-check happens at execution (drill found only the action's own check) [✓/✗].
- **"How do you audit an agent's write?"** Who requested, the approved plan (ID + version), who approved, the executed action with inputs/outputs, the identity it ran under, timestamps — ideally append-only.

- **"Multi-tenant leakage — where could it happen?"** Shared caches keyed without org; shared indexes without tenant filters; logs mixing tenants; a coarse MCP session not bound to org. Your system keys MCP sessions by org and reads org from the user context.

Part 3 — The Grill

How to use it. Cover the answer, say yours out loud (time yourself — 60–90 s), then compare. Each question shows:

- **Testing** — what he's really checking.
- **Answer** — a model spoken answer (adapt "I/we" to your truth).
- **Follow-up** — the next push, with an answer.
- **Trap** — the mistake that loses the point.

Rounds get harder: basics → mechanics → what breaks → design judgement.

Round 1 — The loop and where it runs

Q1. "You've added a new step to the agent's graph. Walk me through how that change gets from your laptop to a customer's org — and how you'd pull it back."

Testing: whether you know your release path end to end, and decouple deploy from release.

Answer: "The graph is a YAML file in the Core codebase. Before I can even push, a pre-push hook checks the graph's invariants and blocks the push if I break one. Then the normal Core path: pre-merge CI, review, merge into the agent's development branch, post-commit unit and functional tests, and I'd run our eval harness against a test org. My understanding is graph changes are promoted weekly — the validated graph becomes the live version and the previous one becomes 'Stable'. It ships inside the Core release, and it's turned on per org with gates. To pull it back for an org, we flip a gate that makes the client load the Stable graph instead — no deploy."

Follow-up: "Does that roll back everything?" → "No — only the graph. Actions, schemas and skill docs ship on the Core train separately, so the graph has to stay compatible with both old and new actions. That's the main weakness."

Trap: inventing canary percentages or bake times for the graph — those were not found.

Q2. "Where does each part of your agent run?"

Testing: a clear mental map.

Answer: "Two places. The definition — graph, skill documents, actions that write to the database — lives in the Core monolith and ships with Core. The runtime is microservices on our internal Kubernetes platform on AWS: an Agent API that keeps sessions in Redis, a worker, a Java planner service and a Python reasoning service that executes the graph, plus an LLM gateway in front of the model providers and a retrieval gateway in front of the Data Cloud knowledge index. Calls into Core for actions go over mTLS."

Follow-up: "Why does the reasoning service need to be Python?" → "The graph runtime and most of the LLM tooling ecosystem are Python; the async model fits an I/O-bound workload."

Trap: blurring which parts are platform and which are yours.

Q3. "Why split the definition into the monolith but the runtime into microservices?"

Testing: architectural reasoning.

Answer: "Actions write Core data, so they belong next to the data model, the permission system and Core's transactions — and they inherit Core's per-org gating and release discipline. The runtime has a completely different profile: long, I/O-bound turns, streaming, model calls, independent scaling and much faster deploy cadence. Putting it in separate services lets it scale and ship on its own schedule without touching Core."

Follow-up: "What does that split cost you?" → "Two release lanes that must stay compatible, cross-service latency, and observability across a boundary — one request touches both worlds."

Trap: "microservices are better" without the cost.

Q4. "What exactly did you build, and what did the platform give you?"

Testing: ownership honesty. This decides whether he trusts the rest.

Answer: "I owned the agent loop: [name your parts — the graph and its states, the skill documents, the access-management actions, the approval flow, the eval suites]. It runs on platform pieces I consumed: the Kubernetes platform and pipelines, the LLM gateway, the retrieval gateway and Data Cloud, and the planner runtime. So when we talk deployment I'll be precise about which configs were mine and which were platform defaults."

Follow-up: "So you didn't run the clusters?" → "No — the platform team does. I can tell you how our workload behaves on them and how I'd tune it."

Trap: claiming the platform. One follow-up exposes it — the CrowdStrike lesson.

Q5. "Draw one turn's request path. Where is it synchronous and where is it async?"

Testing: systems thinking.

Answer: "UI to Agent API is HTTP, and the response streams back as server-sent events. Agent API to worker to planner to reasoner is request/response with streaming. Inside the turn the reasoner calls the retrieval gateway and the LLM gateway synchronously, and calls Core actions synchronously over mTLS — the action runs in-process Java in Core. Asynchronous side paths: action-failure events and gateway feedback and audit go out on Kafka. And approval is the big async boundary: the turn ends when the plan is shown, and the user's approval arrives as a brand-new turn."

Follow-up: "What does that approval boundary do to your observability?" → "It splits one logical request into two traces — Q36."

Trap: forgetting that approval is a separate request.

Round 2 — Containers and runtime

Q6. "What's in your container image, and why?"

Testing: Docker fundamentals applied.

Answer: "For the reasoning service: the corporate RHEL 9 minimal Python 3.12 base image — a platform team patches it centrally, and minimal means fewer CVEs. Dependencies are installed with `uv` from a matching build image, so no compilers end up in the runtime image. It runs as a non-root user, exposes one port, and starts gunicorn."

Follow-up: "Why order Dockerfile steps carefully?" → "Layer caching: dependencies before source, so a code change doesn't reinstall everything."

Trap: describing a generic Dockerfile with no connection to yours.

Q7. "How do you know production runs the image you tested?"

Testing: supply-chain discipline.

Answer: "Build once, promote the same artifact: an immutable reference — the digest or a never-reused tag like the git SHA — moves from staging to production; it's never rebuilt per environment. Stronger still: sign the image in CI and verify the signature at admission to the cluster. I haven't confirmed whether our pipeline verifies signatures at admission, so I won't claim it."

Follow-up: "What's wrong with `latest`?" → "Mutable — two nodes can pull different images under the same tag."

Trap: saying "we tag it latest".

Q8. "Explain your service's concurrency model."

Testing: runtime depth.

Answer: "Gunicorn as the process manager with async Uvicorn workers — six by default. Processes give us all the CPU cores despite Python's GIL; inside each process an event loop holds many turns at once, because a turn is mostly waiting on the model. So one worker can hold dozens to hundreds of concurrent turns; the limits are memory, connections and downstream rate limits, not CPU."

Follow-up: "What kills an async worker?" → "Anything CPU-heavy or blocking on the event loop — a synchronous HTTP call or heavy parsing stalls every turn in that worker. Offload to a thread pool or another service."

Trap: "we use threads".

Q9. "Why would anyone set `preload_app` to False?"

Testing: whether you understand fork semantics — a great differentiator.

Answer: "Preloading loads the app in the gunicorn master and forks workers — saves memory through copy-on-write. But anything holding live OS resources doesn't survive a fork safely: an async HTTP client created before the fork shares sockets and event-loop state across every worker. So we keep it off and each worker builds its own clients after it starts."

Follow-up: "And `max_requests`?" → "5000 with jitter 100 — recycle workers to bound slow memory growth; jitter so they don't all restart together."

Trap: none — this one is pure upside if you know it.

Q10. "What happens to an in-flight agent turn when you deploy?"

Testing: graceful shutdown.

Answer: "Kubernetes removes the pod from the Service, runs the preStop hook and sends SIGTERM. Our worker stops accepting new turns and drains the in-flight ones. The timeouts nest — turn drain deadline, then gunicorn's graceful timeout, then its hard timeout, then Kubernetes' termination grace period, roughly 620 < 640 < 700 < ~900 seconds on our voice path. If any outer number were smaller than an inner one, SIGKILL would land mid-turn and the user's stream would break."

Follow-up: "And for the text stream?" → "Same principle; I haven't verified the exact text-stream behaviour on SIGTERM."

Trap: "Kubernetes handles it" — the default grace is 30 s.

Q11. "Your LLM provider's latency triples. Walk me through what happens."

Testing: reasoning under load — Little's law.

Answer: "By Little's law, concurrency equals arrival rate times latency — so in-flight turns triple at the same traffic. Async absorbs that until we hit memory, connection-pool or worker limits. Time-to-first-token goes up. Then timeouts fire: our MCP proxy waits 60 seconds per attempt with retries, so slow turns get retried — which adds load to an already slow dependency. Mitigations: the gateway's circuit breaker and provider failover, autoscaling on in-flight turns rather than CPU, a retry budget, and shedding load with a clear 'busy, nothing was changed' message."

Follow-up: "Why is CPU-based autoscaling useless here?" → "CPU stays low while workers wait."

Trap: "we'd add more pods" without noticing the bottleneck is downstream.

Q12. "When is it safe to retry?"

Testing: idempotency.

Answer: "When the operation is idempotent, or carries a dedupe key the receiver enforces. Retry only transient failures — timeouts, 5xx, 429 with backoff — never validation errors. Exponential backoff with jitter, a capped number of attempts, and retry at one layer only: three layers each retrying three times is 27 attempts against a struggling dependency. An agent turn that may already have executed a write is not safe to retry blindly."

Follow-up: "How would you make the create-group action idempotent?" → "Look up by name before creating, or key the execution by plan ID + step and record it — Q53."

Trap: "we retry three times" with no idempotency story.

Round 3 — Kubernetes

Q13. "How would you autoscale the reasoning service?"

Testing: applied Kubernetes judgement.

Answer: "On in-flight turns per pod — a custom metric — targeting maybe 60–70% of safe capacity. Min replicas sized so we survive losing an AZ at peak. Scale-down stabilization longer than the longest turn so we don't kill conversations. Memory request equal to limit, sized from per-worker memory times worker count. CPU modest request, generous limit. A PodDisruptionBudget so drains don't take out capacity in bulk."

Follow-up: "How do you get that metric into the HPA?" → "Export it as a Prometheus-style metric and use a custom-metrics adapter, or KEDA."

Trap: "HPA on CPU at 70%".

Q14. "Requests vs limits. OOMKilled vs throttled."

Testing: fundamentals.

Answer: "Request is what the scheduler guarantees and uses for placement; limit is the ceiling. Exceed the CPU limit and you're throttled — slower, shows as latency. Exceed memory and the container is OOMKilled and restarted. Request equal to limit gives Guaranteed QoS — last to be evicted."

Follow-up: "Why might you set no CPU limit?" → "Throttling adds latency to a latency-sensitive path; with sane requests the scheduler still protects neighbours."

Trap: mixing the two up.

Q15. "Give me a probe design for your service."

Testing: whether you know the classic failure.

Answer: "Liveness: a cheap endpoint proving the event loop responds — no dependency checks, otherwise a downstream blip restarts every pod at once. Readiness: initialised, clients created, and false as soon as we receive SIGTERM so new traffic stops while we drain. A startup probe if initialisation is slow, so liveness doesn't kill a pod that's still booting."

Follow-up: "Should readiness check Redis?" → "If the service can't do anything useful without it, yes — it removes the pod from traffic without restarting it. Never in liveness."

Trap: a liveness probe that calls the LLM gateway.

Q16. "How does a rolling update avoid dropping requests?"

Answer: "`maxSurge` adds new pods before old ones go; `maxUnavailable` keeps capacity. New pods take traffic only when readiness passes. Old pods are removed from endpoints, a preStop sleep covers propagation delay, then SIGTERM triggers the drain within the grace period."

Follow-up: "Why the preStop sleep?" → "Endpoint removal propagates asynchronously; without it, a few requests still arrive after SIGTERM."

Trap: forgetting the old-pod side.

Q17. "What's a PodDisruptionBudget and why does an agent service care?"

Answer: "It limits *voluntary* disruptions — node drains, cluster upgrades — to keep at least N pods (or at most M unavailable). Agent turns last minutes; without a PDB an upgrade could evict many pods at once and cut many conversations. With a PDB plus a long grace period, evictions trickle and each pod drains."

Trap: thinking a PDB protects against node crashes (it doesn't — those are involuntary).

Q18. "What does the service mesh give you, and what does it cost?"

Answer: "Envoy sidecars give mTLS and service identity between services, uniform telemetry, network-level retries and timeouts, and traffic shifting — without app changes. Costs: an extra hop of latency, complexity, and a subtle one — mesh retries on top of app retries multiply attempts. In our case, within a domain plain HTTP is upgraded to mTLS by the mesh; across domains traffic goes through gateways."

Trap: "the mesh makes it secure" — it secures *service* identity, not user identity.

Q19. "Helm values are layered. How do you avoid configuration drift?"

Answer: "All config in Git, layered base → environment → instance → domain, changed only through reviewed PRs, and applied only by the pipeline — GitOps, with drift detection. One of our services had platform config applied by hand with a CLI outside CI — exactly the drift risk; I'd move it into the pipeline."

Trap: none if you name the concrete example.

Round 4 — AWS

Q20. "Why Kubernetes and not Lambda for this?"

Answer: "Turns run for minutes and stream tokens — Lambda caps at 15 minutes, cold starts hurt a chat path, streaming through API Gateway is awkward with its ~29-second default integration timeout, and we need a mesh for mTLS. And a shared Kubernetes platform already exists, so the ops cost is amortised. Lambda would fit spiky, short, event-driven side jobs — like reacting to an ingestion event."

Follow-up: "And ECS?" → "Viable for a simple service; the platform standard and mesh decided it."

Trap: "Kubernetes is more scalable" (Lambda scales fine — wrong reason).

Q21. "How does a pod get AWS permissions, and how do you scope them?"

Answer: "Per-pod IAM roles — IRSA or EKS Pod Identity: the service account maps to an IAM role, the pod gets a web-identity token and exchanges it with STS for temporary credentials. Each service gets its own role with least privilege — e.g., read-only on one S3 prefix. On our platform this is platform-managed; I haven't written those roles myself."

Trap: "we use access keys in env vars".

Q22. "How do you keep model traffic and data in-region?"

Answer: "Each org's AI region comes from its provisioned instance. The LLM gateway enforces in-region Bedrock and routes other providers by region; Bedrock via VPC endpoints keeps traffic on the AWS network. The tension: there's a kill switch that can route to another region during an outage — for regulated orgs that's a residency violation, so the policy should be per-org: fail closed rather than fail over cross-region."

Trap: ignoring the kill-switch trade-off.

Q23. "A whole region goes down. What happens to your agent?"

Answer: "Core instances in other regions keep working. Model calls can fail over via the gateway kill switch where policy allows. The real risk is anything single-region and shared — my understanding is the knowledge index lives in one region, so knowledge retrieval would degrade for every org. Mitigations: replicate the index across regions, and degrade gracefully — answer what we can without knowledge and say so, never guess."

Trap: "it's multi-AZ so we're fine" — AZ ≠ region.

Q24. "Why Redis, S3 and Kafka — what does each do for you?"

Answer: "Redis is fast, TTL'd session state between turns — not a source of truth. S3 holds the knowledge corpus files that feed the Data Cloud index, and deferred audit archives — durable and cheap. Kafka carries asynchronous events — gateway feedback and audit, action-failure events — durable, ordered per partition, replayable, so consumers can be added without touching producers."

Trap: using Redis as the durable home of approvals.

Q25. "Tell me about something you configured on AWS yourself."

Testing: the CrowdStrike question. Hands-on or not.

Answer (if you built AdaptiveRAG yourself): "On a side platform I built a Lambda + API Gateway retrieval service: I sized memory — which also sets CPU — to keep FAISS index loading fast, dealt with cold starts by [your fix], wrote the execution role's policy for one S3 bucket, and hit API Gateway's integration timeout on long LLM calls, which pushed me to [async/streaming]. At Salesforce the AWS layer is platform-owned, so my hands-on there is the service layer."

Answer (if not): "At Salesforce the AWS layer is owned by the platform team, so I haven't configured IAM or VPCs there myself. What I own is how our services behave on it — and I can reason through any of those choices with you."

Trap: describing platform work as yours.

Round 5 — CI/CD

Q26. "Walk me through your pipeline, stage by stage."

Answer: "Two lanes. Agent definitions in Core: pre-push invariant hook → pre-merge CI → review → post-commit unit, functional tests and coverage → Core build → release train → per-org gates; graph promoted weekly with a Stable fallback. Runtime services: CI builds a container image → registry → Helm and Terraform deploy → staged rollout across instances with soak and health analysis → rollback. Knowledge content has its own manual path: upload, index rebuild, then a Core version bump."

Follow-up: "Where are evals in that?" → "Run, but advisory — Q32."

Trap: describing a GitHub Actions pipeline you don't have.

Q27. "Canary, blue/green or rolling — which for your services and why?"

Answer: "Stateless runtime services: canary with automated analysis — small blast radius, data-driven. Blue/green where mixed versions are dangerous or instant switch-back matters — the planner uses a blue/green-style release. Rolling is the default fallback. For the graph we effectively do a tenant-based canary through per-org gates."

Trap: not tying the choice to properties of the service.

Q28. "What problem do feature flags solve, and what do they cost?"

Answer: "They separate deploying from releasing: code reaches production off, then turns on per org or per percentage, with a kill switch that doesn't need a deploy. Our Stable-graph rollback is a flag. Costs: combinatorial states to test, flag debt, and flags as hidden config that must be audited."

Q29. "Your rollback only reverts the graph. Why is that a problem and what would you do?"

Answer: "Because a release changes several things — graph, actions, schemas, skill docs — and only one of them can be reverted per org. If a bad skill doc or action ships, the gate doesn't help; you wait for the next Core patch. Short term: expand/contract so the graph always works with both old and new actions. Longer term: version skills together with the graph so rollback is one pointer move, and give actions their own per-org gate."

Trap: not knowing why it matters.

Q30. "How do you ship a model upgrade safely?"

Answer: "Treat it as a release. Offline: run the full eval suite on the new model vs the current one — tool selection, arguments, safety cases at 100%, judged quality, latency and cost. Then canary to a small set of orgs, compare online metrics, expand. Keep the old model behind a flag. Watch regional availability: if the new model isn't available in-region for some orgs, those orgs need per-org routing, not a global switch."

Trap: "we just change the model ID".

Q31. "You changed a tool's input schema. A plan approved yesterday executes today. What happens?"

Answer: "It may call the tool with the old shape and fail — or worse, succeed with the wrong meaning. Fixes: version tool schemas and record the version in the plan; accept both shapes during transition; re-validate the plan at execution and, if it no longer validates, ask the user to re-approve rather than guessing."

Q32. "Should evals block releases?"

Answer: "Deterministic and safety checks — yes: right tool, right arguments, permission check fired, approval before every write; safety at 100%, no regression vs the stable baseline. LLM-judged quality — not yet: our judge's agreement with humans is about kappa 0.65, so it should warn, not block, until it's re-calibrated. Today ours are advisory; gating the weekly promotion on the deterministic set is what I'd change first."

Q33. "One service's config is applied by hand. So what?"

Answer: "Then Git doesn't describe production. A rollback of the code won't roll back the config; two engineers can overwrite each other; an incident review can't tell what changed. Move it into the pipeline and alert on drift."

Round 6 — Observability and on-call

Q34. "A user says the agent replied 'Something went wrong'. Debug it."

Answer: "Session ID and time first. Core logs for that session: the LLM-timing records tell me which model call was slow or failed. Then the planner's step trace: did an LLM step, a tool call or the grounding check fail? The action record gives status, latency and error code. Empty grounding → retriever logs and the gateway dashboard. Then compare the turn's duration with the timeout chain — we've seen long turns show this error even though the change succeeded, because an outer layer gave up first. Finally: what changed — deploy, gate, graph promotion, model?"

Follow-up: "It succeeded but showed an error — what do you tell the user?" → "Read back the actual state before telling the user anything; never claim failure without checking."

Q35. "What's in your logs? Show me a log line."

Answer:

```
{"ts":"…","service":"core","event":"action","action":"CreateGroup",
 "status":"SUCCESS","timeMs":412,"targetId":"…","traceId":"…",
 "orgId":"…","sessionId":"…","planId":"…"}
```

"Structured, with trace, org, session and plan IDs so I can pivot. LLM-timing records carry template, generation ID, durations and token counts. No secrets, no raw PII."

Trap: free-text logs.

Q36. "How do traces work across your services, and where do they break?"

Answer: "A trace ID travels in headers and each service adds spans. It breaks at four places: async hops, unless the ID is put in the message; long-lived streams; third-party hops like Slack or an MCP client; and above all the approval pause — the approval comes back as a new request with a new trace. So I'd add a business correlation key — session ID plus plan ID — on every span and log line, alongside org and user ID, independent of the trace ID."

Q37. "Head-based or tail-based sampling for agents?"

Answer: "Tail. Failures and slow turns are rare and they're what you need; head-based at 1% throws away 99% of them. Tail sampling keeps all errors and slow traces and samples the healthy rest — at the cost of a collector tier that buffers traces."

Q38. "What SLOs would you set for this agent, and how would you alert?"

Answer (as proposals): "Turn success: 99.5% of turns complete without a system error over 30 days. Responsiveness: time-to-first-token p95 under 3 seconds — that's where users decide the agent is dead. Write correctness: 99.9% of approved actions either succeed or fail cleanly with no partial state. Alert with multi-window burn rates — page at ~14× over an hour and five minutes, ticket at ~1× over three days."

Follow-up: "Do you have these today?" → "No — none were found. These are what I'd propose."

Trap: presenting proposals as existing targets.

Q39. "What pages someone at 3 a.m.?"

Answer: "User-facing symptoms: turn-success burn rate, time-to-first-token breach, an action-failure spike, and LLM-gateway or retrieval availability when they're on the critical path. Not CPU, not a single pod restart. Every page links a runbook."

Q40. "How do you know the agent is getting *worse* when nothing is erroring?"

Answer: "Quality signals in production: grounding-failure rate per response, tool-selection accuracy from online evals on sampled traces, clarification rate, plan-rejection rate, users re-asking the same thing. Track them per graph version and per model, so a promotion that degrades quality shows up as a step change."

Q41. "Tell me about an incident."

Answer (only as yours if you were involved): "A data-query action started returning 500s; users saw a generic apology. The root cause was a dependency: the egress mesh couldn't create request context for calls to an API-protection service, three retries hit the same failure, then a 500. It self-resolved within hours. Lessons: retries don't help when the dependency is down — classify and break the circuit; tell the user 'a dependency is unavailable, nothing was changed' instead of apologising; and alert on dependency error rate, not only our own 500s."

Trap: claiming you resolved it if you didn't.

Q42. "Walk me through the first 15 minutes of a page."

Answer: "Acknowledge. Top-row dashboard: is it all orgs or some, one region or all, since when? Check recent changes — deploys, gates, graph promotion, model. Mitigate before root-causing: roll back, flip the gate to Stable, fail over the provider, shed load. Post a status update with scope and next update time. Then dig in, and afterwards a blameless postmortem with an eval case added."

Q43. "Your retriever alerts go to Slack only. Defend it or fix it."

Answer: "Defensible if retrieval outages degrade gracefully and another page covers the user impact. But the agent's first node is knowledge retrieval, so a retrieval outage is effectively an agent outage. I'd page on the user-facing symptom — turn-success burn — so whoever's on call is woken regardless of which dependency failed, and route Sev1 retrieval availability to the pager with a clear owner."

Round 7 — Tools, MCP and evals

Q44. "How does the agent pick the right tool out of hundreds?"

Answer: "Retrieval, not a giant list. System primitives come from a semantic search over tool descriptions — about 50 candidates, keep those with similarity at least 0.7, then the top two. If no skill matches, a global vector fallback offers up to five per goal. Then the executor only sees tools enabled for its current state, and every argument is validated against the tool's JSON schema."

Follow-up: "How did you pick 0.7?" → "Tune on a labelled set: plot whether the correct tool appears vs threshold, pick the knee, re-tune when the catalogue changes."

Trap: "top-5 at 0.7" — that's two different settings.

Q45. "Why expose the whole agent as one MCP tool?"

Answer: "Safety. External agents get a conversation, not raw write primitives — so our plan approval, confirmation and access checks always run and can't be bypassed. Costs: latency — p50 around 35 s, p95 around 98 s in a local sample — and less composability. I'd expose read-only operations as fine-grained tools marked read-only and keep writes behind the agent."

Q46. "How does an MCP call carry the user and the tenant?"

Answer: "The session is keyed on org, MCP session and bot, and the proxy passes caller context into the agent, so tenant is in the key and every action still runs Core's own access check. The exact field carrying the user I'd want to confirm. The right design is an audience-bound OAuth token for that user, exchanged for downstream calls — never passed through, which the MCP spec forbids."

Q47. "Your LLM judge has kappa 0.65. Can it gate releases?"

Answer: "0.65 is the low end of 'substantial' agreement — fine to inform, risky as the sole gate: it would block good releases and pass bad ones often enough to erode trust. Gate on deterministic checks; keep judged scores as warnings; improve the judge — clearer rubric, anchored examples, a different model family — and re-audit."

Q48. "How does a production failure become an eval case?"

Answer: "Trace from the incident → label the root-cause layer (retrieval, planning, tool, auth, infra) → write the minimal reproducing utterance with an org fixture → write a deterministic expected behaviour → add it to the regression suite tagged with the incident → fix → it passes and stays forever. Our harness supports this, but the path isn't written down as a process — that's one I'd formalise."

Round 8 — Identity, data and design curveballs

Q49. "How do you guarantee the agent can't do something the user can't?"

Answer: "Two identities. mTLS proves the service — in production there's no fallback; a missing certificate fails fast. The user travels as context, Core rebuilds the user context, and the action's own access check plus the skill's declared permissions run for the admin who asked. The model never decides authorization: it only sees tools enabled for its current state, and the action re-checks in code."

Trap: saying it as fact before you've confirmed which user ID Core receives.

Q50. "What stops a compromised internal service from asserting any user?"

Answer: "Honestly, in a header-based design, mostly the mesh: Core trusts authenticated services to assert the user — a confused-deputy risk. Mitigations: a tight allowlist of which services may assert user context, and binding the user cryptographically — an on-behalf-of token exchange, where the downstream token names both the user and the acting service, is audience-bound and short-lived."

Q51. "Redis fails over while the admin is looking at the plan. What happens?"

Answer: "If the session or graph state is lost, the next message starts fresh and the earlier approval can't be applied — the user has to start over. I'd persist the plan and approval durably, keyed by plan ID — a database or checkpointer — with Redis as a cache in front. Approval should be bound to the plan's hash and version, expire, and be re-validated on resume."

Q52. "Two admins edit the same group at the same time — or one double-clicks Approve."

Answer: "Double-click: idempotency key on plan ID + step + version; the first execution records it, the second returns the first result. Concurrent edits: optimistic concurrency — capture the group's version at plan time, re-check at execution, and fail closed with 'this changed since you approved'. In Salesforce you'll also see row-lock contention on group membership — bounded retries with backoff."

Q53. "The group was created, the membership step failed. Now what?"

Answer: "Two actions, two commits, no transaction across them — so an empty group exists. Make create idempotent by name so a retry doesn't fail on a duplicate or create a second group. Treat group creation as the pivot step: after it, only go forward — retry membership, which is naturally idempotent. Compensate by deleting the empty group only if the user abandons. And report the partial state truthfully in the task list."

Q54. "Design an approval that survives hours and restarts."

Answer: "This is what I built in ResolveFlow: LangGraph interrupts pause the graph, and a Postgres checkpointer persists state at every step keyed by thread ID — the run survives restarts and resumes exactly where it paused. On top: the approval record binds to the plan hash, has an expiry, resume is idempotent, and on resume we re-validate permissions and state before executing. The write runs under the approving user's own GitHub OAuth token."

Q55. "Where could tenant data leak in your design?"

Answer: "Shared caches keyed without the org; a shared index without a tenant filter; logs and traces mixing tenants that the wrong people can query; a coarse MCP session not bound to an org; eval sets built from production data. Our MCP sessions are keyed by org, Core reads the org from the user context, and the knowledge index holds product documentation, not customer data — customer data only comes from live Core reads under the user's context."

Q56. "'Throughput up 40%' — how did you measure that?"

Answer template: "It's [completed executions per hour / requests per second] at [the same replica count], measured from [dashboard/report] comparing [window A] with [window B] under [similar load]. The change was [bulkheads per workflow / async / right-sizing]. The caveat is [what else changed]."

If you can't fill it: "I'd rather give you the mechanism than a number I can't back: we isolated workflows into separate worker pools, so one slow downstream no longer starved the others, and moved blocking calls to async."

Trap: any number without a method.

Q57. "One quarter to harden this platform. What do you do?"

Answer: "Six things, in order. One: a correlation key across the approval pause, plus SLOs and burn-rate paging — today outages can be found by customers first. Two: gate the weekly graph promotion on deterministic and safety evals. Three: durable approval state, idempotent execution and re-validation at execute time. Four: one fail-closed source of truth for confirmation flags. Five: version skills with the graph so rollback is complete. Six: all config applied by the pipeline."

Q58. "How would you build the GitLab version: an agent that fixes a failing pipeline?"

Testing: transfer to his domain.

Answer: "Same loop. Trigger: the pipeline-failed event. Ground: job logs, the diff, recent commits, and similar past failures through retrieval. Route: deterministic handling for known classes — retry a flaky job, quarantine a known-flaky test — before any LLM. Plan: the LLM proposes a fix as a merge request, never a push to the main branch. Gate: the pipeline itself re-runs on the MR and is the verifier, and a human approves. Act: under a least-privilege identity on behalf of the triggering user. Verify and learn: post-merge validation, and every failure that reached a human becomes an eval case. Observability: a trace per remediation with project, tenant and user IDs — the same loop I built in ResolveFlow for GitHub."

Appendix — Rapid-fire glossary

Term	One line
Container	A Linux process isolated by namespaces (visibility) and cgroups (resources)
Image / layer	Read-only filesystem template built in cached layers
Digest	Immutable content hash of an image

Term	One line
Multi-stage build	Build in one stage, ship only artefacts in a clean runtime stage
Pod	Smallest Kubernetes unit; containers sharing IP and volumes
Deployment	Keeps N replicas and rolls out changes
Service	Stable DNS name load-balancing to ready pods
Request / limit	Scheduling guarantee / hard ceiling
OOMKilled	Memory over limit → container killed
Liveness / readiness	Restart if stuck / remove from traffic if not ready
HPA	Scales replicas on a metric
PDB	Caps voluntary disruptions
Helm	Charts = templates + layered values
Service mesh	Sidecar proxies giving mTLS, telemetry, retries, traffic shifting
GitOps	Git is the desired state; a controller syncs and flags drift
Canary	Small % first, analyse, expand or roll back
Blue/green	Full parallel version, switch traffic at once
Feature flag	Deploy off, release later; kill switch
Expand/contract	Add new → migrate → remove old; keeps separately-shipped parts compatible
IRSA	Kubernetes service account → IAM role, per pod
PrivateLink	Private access to a service without the internet
RPO / RTO	Data you can lose / time you can be down
Terraform state	Record of what Terraform manages; remote + locked
TTL	Expiry time on a key
Eviction policy	What Redis drops when memory is full
Isolation level	How much concurrent transactions see of each other
Lost update / write skew	Concurrency anomalies under weak isolation
Optimistic concurrency	Version check on write (compare-and-set)
At-least-once	May duplicate; needs idempotent consumers
Transactional outbox	Write row + event in one transaction; relay publishes
DLQ	Where repeatedly failing messages go
Structured log	Key-value log line you can query
Span / trace	One operation / the tree of operations for a request
traceparent	W3C header carrying trace and parent span IDs
Head / tail sampling	Decide at start / decide after the trace completes

Term	One line
Cardinality	Number of unique label combinations of a metric
SLI / SLO / SLA	Measurement / target / contract
Error budget	Allowed failure under the SLO (99.9%/30 d = 43.2 min)
Burn rate	Speed of spending the error budget
Golden signals	Latency, traffic, errors, saturation
Little's law	In-flight = arrival rate × latency
Circuit breaker	Stop calling a failing dependency for a while
Bulkhead	Separate pools so one failure can't exhaust all capacity
Idempotency key	Dedupe marker so a repeated request has one effect
mTLS	Both sides present certificates — service identity
Confused deputy	A trusted component misused to act for someone unauthorised
Token exchange / OBO	Swap a user token for an audience-bound token naming user + actor
MCP	Open protocol: hosts/clients/servers, JSON-RPC, tools/resources/prompts
Cohen's kappa	Agreement beyond chance; 0.61–0.80 = substantial